

UNIVERSIDADE ESTADUAL DO OESTE DO PARANÁ - UNIOESTE

CENTRO DE CIÊNCIAS EXATAS E TECNOLÓGICAS

Colegiado de Ciência da Computação

Curso de Bacharelado em Ciência da Computação

Da Gramática à Execução: Projeto e Implementação de uma Linguagem de Programação, Compilador e Máquina Virtual com Fins Didáticos

Trabalho de Conclusão de Curso

Pedro Henrique de Oliveira Berti



Cascavel–PR

2026

UNIVERSIDADE ESTADUAL DO OESTE DO PARANÁ - UNIOESTE

CENTRO DE CIÊNCIAS EXATAS E TECNOLÓGICAS

Colegiado de Ciência da Computação

Curso de Bacharelado em Ciência da Computação

Pedro Henrique de Oliveira Berti

Da Gramática à Execução: Projeto e Implementação de uma Linguagem de Programação, Compilador e Máquina Virtual com Fins Didáticos

Monografia apresentada como requisito parcial para obtenção do grau de Bacharel em Ciência da Computação, do Centro de Ciências Exatas e Tecnológicas da Universidade Estadual do Oeste do Paraná - Campus de Cascavel.

Orientador(a): Guilherme Galante

Cascavel–PR

2026

PEDRO HENRIQUE DE OLIVEIRA BERTI

DA GRAMÁTICA À EXECUÇÃO: PROJETO E IMPLEMENTAÇÃO DE UMA LINGUAGEM DE PROGRAMAÇÃO, COMPILADOR E MÁQUINA VIRTUAL COM FINS DIDÁTICOS

Monografia apresentada como requisito parcial para obtenção do Título de Bacharel em Ciência da Computação, pela Universidade Estadual do Oeste do Paraná, Campus de Cascavel, aprovada pela Comissão formada pelos professores:

Prof. Guilherme Galante (Orientador)
Colegiado de Ciência da Computação, UNIOESTE

Prof. André Luiz Brum
Colegiado de Ciência da Computação, UNIOESTE

Prof. Anibal Mantovani Diniz
Colegiado de Ciência da Computação, UNIOESTE

Resumo

Na disciplina de Compiladores, o código intermediário gerado pelo compilador só pode ser executado com o uso de uma ferramenta externa, o que deixa o estudante sem ver o resultado final do processo, da gramática à execução. Este trabalho apresenta o projeto e a implementação de uma linguagem de programação com sintaxe próxima ao português, denominada Bertol, acompanhada de um compilador completo e de uma Máquina Virtual, denominada Berti VM, desenvolvidos com fins didáticos para apoio ao ensino dessa disciplina. O objetivo é oferecer uma ferramenta educacional integrada que mostre, de forma detalhada e interativa, todas as etapas da compilação, da análise léxica à execução do código gerado. O compilador foi implementado inteiramente em JavaScript puro, sem dependências externas e sem ferramentas geradoras automáticas, executando diretamente no navegador. Ele contempla quatro fases: análise léxica, realizada por um Autômato Finito Determinístico com 22 estados; análise sintática, implementada por dois analisadores independentes, pelos métodos SLR e LL(1); análise semântica, com 22 regras formalizadas e tabela de símbolos; e geração de Código de Três Endereços (TAC). Cada fase expõe ao usuário sua trajetória interna, permitindo a visualização passo a passo dos estados do autômato, dos movimentos da pilha sintática e das verificações semânticas, com modos de operação com e sem recuperação de erros. A Berti VM interpreta e executa o TAC gerado, com execução passo a passo, contínua ou instantânea, exibindo memória, entrada e saída durante a execução, e aceita também o formato genérico de TAC com vetores, ponteiros e funções, de modo que pode executar o código intermediário de outras linguagens desenvolvidas em sala de aula. A contribuição principal é didática: o trabalho mostra todo o processo de compilação de forma detalhada e, com a Berti VM, permite ao estudante executar o código que o seu próprio compilador produz. O sistema foi validado com 14 programas válidos e 11 programas com erros intencionais no compilador, com dez programas executados na Berti VM e com cinco programas em TAC para os recursos de memória indexada e chamadas de função. Todos os testes foram concluídos com sucesso, e as execuções na Berti VM produziram as saídas esperadas nos três modos de execução.

Palavras-chave: compiladores, linguagem de programação, análise léxica, análise sintática, análise semântica, código de três endereços, máquina virtual, ferramenta didática.

Lista de figuras

Figura 1 – Fases de um compilador 15
Figura 2 – Autômato Finito Determinístico do analisador léxico da linguagem Bertol . 25

Lista de tabelas

Tabela 1 – Comparação entre ferramentas educacionais relacionadas a código intermediário 22
Tabela 2 – Categorias de tokens da linguagem Bertol e suas expressões regulares . . . 26
Tabela 3 – Regras semânticas da linguagem Bertol 35
Tabela 4 – Tabela de compatibilidade de tipos da linguagem Bertol 37
Tabela 5 – Padrões de tokens e regras semânticas associadas 37
Tabela 6 – Tradução linha a linha do Exemplo 1 47
Tabela 7 – Tradução linha a linha do Exemplo 2 49
Tabela 8 – Tradução linha a linha do Exemplo 3 51
Tabela 9 – Instruções de atribuição e de operação 54
Tabela 10 – Instruções de memória indexada: vetores e ponteiros 55
Tabela 11 – Instruções de desvio 55
Tabela 12 – Instruções de entrada, saída e funções 56
Tabela 13 – Operadores binários aceitos 56
Tabela 14 – Conversões nomeadas aceitas 57
Tabela 15 – Tipos internos da Berti VM 58
Tabela 16 – Erros detectados pela Berti VM 63
Tabela 17 – Execução passo a passo do programa com chamada de função 65
Tabela 18 – Execução passo a passo do programa com vetor e ponteiro 66
Tabela 19 – Execução do Código 2 na Berti VM 71
Tabela 20 – Programas de teste 1 a 7 e construções da linguagem exercitadas 72
Tabela 21 – Programas de teste 8 a 14 e construções da linguagem exercitadas 73
Tabela 22 – Resultado dos programas com erros intencionais 87
Tabela 23 – Execução dos programas de teste 1 a 3 na Berti VM 88
Tabela 24 – Execução dos programas de teste 4 e 5 na Berti VM 88
Tabela 25 – Execução dos programas de teste 6 e 7 na Berti VM 89
Tabela 26 – Execução dos programas de teste 8 a 10 na Berti VM 89
Tabela 27 – Resultado dos programas em TAC com memória indexada e funções 92
Tabela 28 – Resumo dos casos de teste e resultados 93

Lista de códigos

Código 1 – Programa exemplo em Bertol 70
Código 2 – Código de Três Endereços gerado a partir do Código 1 71
Código 3 – Programa de teste 1: soma de dois números 73
Código 4 – TAC gerado para o programa de teste 1 73
Código 5 – Programa de teste 2: par ou ímpar 74
Código 6 – TAC gerado para o programa de teste 2 74
Código 7 – Programa de teste 3: maior de dois números 74
Código 8 – TAC gerado para o programa de teste 3 75
Código 9 – Programa de teste 4: número dentro de um intervalo 75
Código 10 – TAC gerado para o programa de teste 4 76
Código 11 – Programa de teste 5: fatorial 76
Código 12 – TAC gerado para o programa de teste 5 77
Código 13 – Programa de teste 6: Fibonacci até N termos 77
Código 14 – TAC gerado para o programa de teste 6 78
Código 15 – Programa de teste 7: contagem regressiva 78
Código 16 – TAC gerado para o programa de teste 7 79
Código 17 – Programa de teste 8: número primo 79
Código 18 – TAC gerado para o programa de teste 8 80
Código 19 – Programa de teste 9: soma dos dígitos 81
Código 20 – TAC gerado para o programa de teste 9 81
Código 21 – Programa de teste 10: tabuada 81
Código 22 – TAC gerado para o programa de teste 10 82
Código 23 – Programa de teste 11: tipos primitivos 82
Código 24 – TAC gerado para o programa de teste 11 83
Código 25 – Programa de teste 12: expressões com inteiros e decimais 83
Código 26 – TAC gerado para o programa de teste 12 83
Código 27 – Programa de teste 13: operadores lógicos ?? e `XX` 84
Código 28 – TAC gerado para o programa de teste 13 84
Código 29 – Programa de teste 14: comentário e `PARA` com passo 2 84
Código 30 – TAC gerado para o programa de teste 14 85
Código 31 – Programas com erros léxicos e sintáticos (E1 a E5) 86
Código 32 – Programas com erros semânticos (E6 a E11) 86
Código 33 – Programa TAC 1: vetor, preenchimento e soma 90
Código 34 – Programa TAC 2: ponteiros, troca de dois valores 90
Código 35 – Programa TAC 3: função de soma 91
Código 36 – Programa TAC 4: chamadas aninhadas, soma de quadrados 91

Código 37 – Programa TAC 5: vetor e função, maior elemento 92

Lista de abreviaturas e siglas

AFD	Autômato Finito Determinístico
CIL	*Common Intermediate Language*
CLR	*Common Language Runtime*
CSS	*Cascading Style Sheets*
ETAC	*Extended Three-Address Code* (Código de Três Endereços Estendido)
GLC	Gramática Livre de Contexto
HTML	*HyperText Markup Language*
IR	*Intermediate Representation* (Representação Intermediária)
JVM	*Java Virtual Machine* (Máquina Virtual Java)
LALR	*Look-Ahead Left-to-right, Rightmost derivation*
LL(1)	*Left-to-right, Leftmost derivation*, 1 *lookahead*
LLVM	*Low Level Virtual Machine*
LR	*Left-to-right, Rightmost derivation in reverse*
MIPS	*Microprocessor without Interlocked Pipeline Stages*
PC	Contador de Programa (*Program Counter*)
PLY	*Python Lex-Yacc*
RISC	*Reduced Instruction Set Computer*
RS	Regra Semântica
SLR	*Simple LR*
TAC	*Three-Address Code* (Código de Três Endereços)
UNIOESTE	Universidade Estadual do Oeste do Paraná
VM	*Virtual Machine* (Máquina Virtual)

Lista de símbolos

$\rightarrow$	Produção da gramática: o não terminal à esquerda pode ser substituído pela cadeia à direita
$\Rightarrow$	Passo de derivação
|	Alternativa, em produções da gramática e em expressões regulares
$*$	Fecho de Kleene: zero ou mais repetições do elemento anterior
α	Cadeia qualquer de símbolos terminais e não terminais
$\leftarrow$	Atribuição, nos algoritmos de geração de código

Sumário

1 Introdução 12

2 Revisão Bibliográfica 14

2.1 Etapas de Compilação 14

2.2 Código de Três Endereços 17

2.3 Máquinas Virtuais 18

2.4 Trabalhos Relacionados 20

2.5 Diferenças em Relação aos Trabalhos Relacionados 21

3 Metodologia 23

3.1 Contexto e Problema 23

3.2 Visão Geral 24

3.3 Análise Léxica 24

3.4 Análise Sintática 28

3.4.1 Gramática Livre de Contexto 28

3.4.2 Parser SLR 31

3.4.3 Parser LL(1) 32

3.5 Análise Semântica 34

3.5.1 Regras Semânticas 34

3.5.2 Compatibilidade de Tipos 36

3.5.3 Implementação 37

3.5.4 Recuperação de Erros 38

3.6 Geração de Código de Três Endereços 39

3.6.1 Estruturas Internas 39

3.6.2 Geração de Expressões 39

3.6.3 Geração de Estruturas Condicionais 42

3.6.4 Geração de Estruturas de Repetição 43

3.6.5 Tratamento do Comando PARE 45

3.6.6 Geração de Comandos de Entrada e Saída 45

3.6.7 Geração de Declarações e Atribuições 45

3.6.8 Tradução de Bertol para TAC 46

3.7 Berti VM 51

3.7.1 Tecnologias e Organização do Código 52

3.7.2 Formato de Entrada: Especificação das Instruções 52

3.7.2.1 Regras Gerais 52

3.7.2.2 Gramática 53

3.7.2.3 Instruções e Seus Efeitos . . . 54
3.7.3 Modelo de Dados . . . 57
3.7.4 Modelo de Memória . . . 58
3.7.5 Funções, Pilha de Parâmetros e Pilha de Chamadas . . . 59
3.7.6 Ciclo de Execução . . . 60
3.7.6.1 Carga do Programa . . . 60
3.7.6.2 Execução de uma Instrução . . . 60
3.7.7 Entrada e Saída . . . 61
3.7.8 Modos de Execução e Controles . . . 62
3.7.9 Tratamento de Erros . . . 62
3.7.10 Interface . . . 63
3.7.11 Execução Passo a Passo . . . 64
3.8 Validação . . . 66
3.8.1 Validação do Compilador . . . 66
3.8.2 Validação da Berti VM . . . 67
3.8.2.1 Execução do TAC Gerado pelo Compilador . . . 67
3.8.2.2 Execução de Instruções do Formato Genérico . . . 67
3.8.2.3 Critérios de Aceitação . . . 68

4 Resultados . . . 69
4.1 Estado de Implementação . . . 69
4.2 Exemplo de Execução . . . 70
4.3 Validação . . . 72
4.3.1 Validação do Compilador . . . 72
4.3.1.1 Programas Válidos . . . 72
4.3.1.2 Programas com Erros Intencionais . . . 85
4.3.2 Validação da Berti VM . . . 88
4.3.2.1 Execução do TAC Gerado pelo Compilador . . . 88
4.3.2.2 Execução de Instruções do Formato Genérico . . . 90
4.3.2.3 Comparação entre os Modos de Execução . . . 93
4.3.3 Síntese dos Resultados . . . 93

5 Conclusão . . . 94

Referências . . . 96

1 Introdução

Compiladores traduzem código escrito de uma linguagem-fonte para uma linguagem-alvo executável, detectando e reportando erros durante o processo (AHO et al., 2008). Sua construção envolve conhecimento de linguagens formais, estruturas de dados, algoritmos e arquitetura de computadores (COOPER; TORCZON, 2012). Na disciplina de Compiladores do curso de Ciência da Computação da Universidade Estadual do Oeste do Paraná (UNIOESTE), campus Cascavel, o conteúdo abrange análise léxica, análise sintática, análise semântica, geração de código intermediário, otimização de código e geração de código final, embora as duas últimas etapas sejam abordadas apenas no plano teórico, sem implementação prática por parte dos estudantes.

A representação intermediária utilizada nesse contexto é o Código de Três Endereços (TAC, do inglês *Three-Address Code*), uma forma de representação em que cada instrução envolve no máximo três operandos, incluindo atribuições, desvios condicionais e desvios incondicionais (AHO et al., 2008). Contudo, o ciclo prático da disciplina se encerra com a geração desse código, sem que o estudante o execute. Mernik e Žumer (2003) observam que, em disciplinas de construção de compiladores, os estudantes aprendem a escrever compiladores manualmente e a utilizar ferramentas geradoras de analisadores, como *lex* e *yacc*. O *lex* é um gerador de analisadores léxicos, a partir de expressões regulares que descrevem os tokens da linguagem produz automaticamente o código do analisador léxico. O *yacc* (*Yet Another Compiler Compiler*) é um gerador de analisadores sintáticos, a partir de uma gramática livre de contexto produz o código de um analisador ascendente (AHO et al., 2008). Mesmo com o uso dessas ferramentas, a execução do código gerado não faz parte do escopo prático dessas disciplinas.

Além de não contemplarem a execução, as ferramentas tradicionais apresentam outra limitação didática. A literatura aponta que *lex* e *yacc* possuem pouco ou nenhum valor didático, uma vez que abstraem os algoritmos internos e não permitem ao estudante acompanhar o processo de compilação em funcionamento (MERNIK; ZUMER, 2003). Em contrapartida,

Nisan e Schocken (2005) demonstram, no projeto Nand2Tetris, que a construção de um sistema completo, do hardware ao compilador e à máquina virtual, em que o estudante implementa cada camada manualmente, proporciona compreensão mais concreta do que o estudo de cada camada de forma isolada. Na mesma direção, Sebesta (2011) argumenta que o estudo de linguagens de programação é mais efetivo quando o aprendiz tem contato direto com os mecanismos internos de implementação. Nesse contexto, o problema abordado neste trabalho é: como projetar uma linguagem de programação e implementar o ciclo completo de compilação, da análise léxica à execução, de modo que a etapa de execução seja incluída no processo didático?

Para responder a essa questão, este trabalho propõe a construção de três artefatos: a linguagem de programação Bertol, uma linguagem imperativa de tipagem estática com sintaxe em português estruturado, um compilador para essa linguagem, que expõe ao usuário o funcionamento interno de cada fase e uma Máquina Virtual denominada Berti VM, responsável por interpretar e executar o TAC produzido pelo compilador.

Organização dos Capítulos

O Capítulo 2 apresenta a revisão bibliográfica, abordando as etapas de compilação, o Código de Três Endereços, máquinas virtuais e trabalhos relacionados que envolvem geração ou execução de código intermediário com fins educacionais. O Capítulo 3 descreve a metodologia adotada, detalhando a implementação de cada fase do compilador, a implementação da Berti VM e o processo de validação. O Capítulo 4 apresenta os resultados obtidos, incluindo exemplos de execução do compilador e da Berti VM e a validação por meio de casos de teste. O Capítulo 5 conclui o trabalho, discutindo as contribuições e as limitações da solução desenvolvida.

2

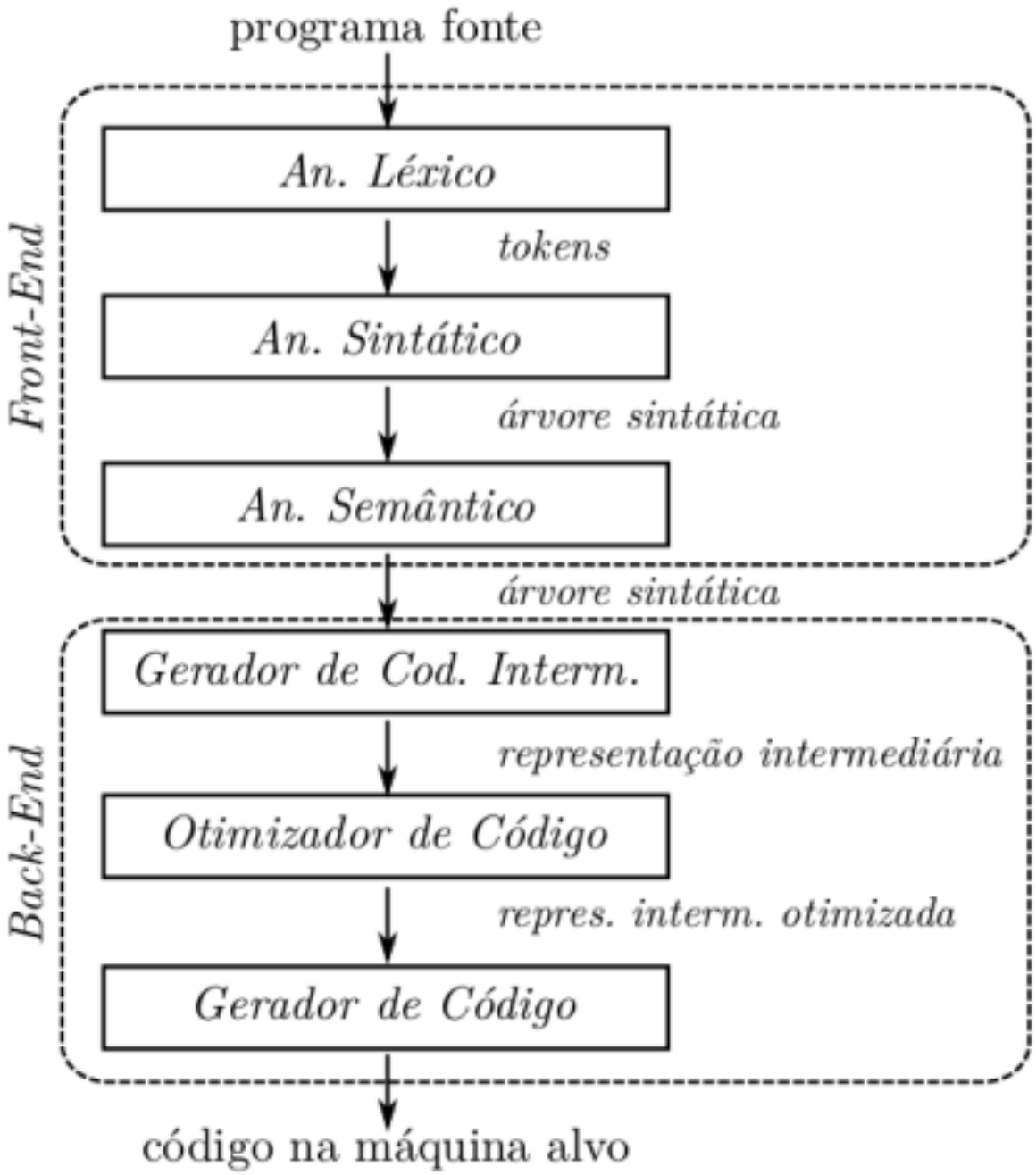
Revisão Bibliográfica

Este capítulo apresenta os fundamentos teóricos utilizados no desenvolvimento deste trabalho. A Seção 2.1 descreve as etapas do processo de compilação, com ênfase na análise léxica, sintática e semântica. A Seção 2.2 aborda representações intermediárias, com foco no Código de Três Endereços. A Seção 2.3 trata de máquinas virtuais. A Seção 2.4 apresenta trabalhos relacionados que envolvem geração ou execução de código intermediário com fins educacionais. A Seção 2.5 descreve as diferenças entre este trabalho e os trabalhos relacionados apresentados.

2.1 Etapas de Compilação

Um compilador converte código-fonte de uma linguagem de alto nível em código de máquina ou em código intermediário. Esse processo é dividido em fases, cada uma com função específica (AHO et al., 2008). A Figura 1 apresenta o esquema geral dessas fases, agrupadas em dois blocos: o *front-end*, que analisa o programa fonte e reúne as análises léxica, sintática e semântica, e o *back-end*, que gera e otimiza o código intermediário e produz o código para a máquina-alvo. Entre uma fase e a seguinte, a figura indica o produto repassado: tokens, árvore sintática, representação intermediária e representação intermediária otimizada. As subseções seguintes descrevem as fases nessa ordem.

Figura 1 – Fases de um compilador



Fonte: Galante (2025)

Conforme a Figura 1, o programa fonte entra no analisador léxico, a primeira fase do *front-end*, que lê o fluxo de caracteres e o agrupa em sequências chamadas lexemas, produzindo como saída os tokens. Para cada lexema, o analisador léxico produz um token na forma $\langle nome, valor \rangle$, em que o nome identifica a categoria do token e o valor aponta para a entrada correspondente na tabela de símbolos (AHO et al., 2008). A tabela de símbolos é a estrutura de dados que armazena as informações sobre os identificadores do programa, como nome, tipo e escopo, e é consultada e atualizada ao longo de várias fases da compilação (AHO et al., 2008). Espaços em

branco e comentários são descartados na análise léxica (SINGH, 2009). Os tokens são definidos por expressões regulares, uma notação formal que descreve conjuntos de cadeias por meio de concatenação, alternância e repetição. Por exemplo, a expressão *letra* (*letra* | *dígito*)* descreve identificadores formados por uma letra seguida de zero ou mais letras ou dígitos (AHO et al., 2008). A abordagem usual para construir um analisador léxico é implementar um Autômato Finito Determinístico (AFD) que reconhece as linguagens regulares correspondentes a cada definição de token (MOGENSEN, 2010).

Seguindo a Figura 1, os tokens são entregues à análise sintática, que verifica se a sequência está em conformidade com a gramática formal da linguagem e produz como saída a árvore sintática, uma estrutura hierárquica que organiza os tokens (SINGH, 2009). A árvore sintática explicita a relação entre os operadores e operandos de cada construção da linguagem, e é utilizada como entrada para as fases subsequentes (SUNITHA, 2013). A sintaxe das linguagens de programação é, em geral, especificada por uma gramática livre de contexto (GLC), formada por um conjunto de símbolos terminais (os tokens), um conjunto de símbolos não terminais, um símbolo inicial e um conjunto de produções da forma $A \to \alpha$, em que A é um não terminal e α é uma sequência de terminais e não terminais (AHO et al., 2008). Uma derivação é a aplicação sucessiva de produções, partindo do símbolo inicial, até se obter a cadeia de tokens. Quando, em cada passo, o não terminal substituído é sempre o mais à esquerda, a derivação é chamada de derivação mais à esquerda; quando é sempre o mais à direita, é chamada de derivação mais à direita (AHO et al., 2008). Considere, por exemplo, a gramática com as produções $E \to E + T \mid T$ e $T \to \text{id}$, e a cadeia id + id. A derivação mais à esquerda é $E \Rightarrow E + T \Rightarrow T + T \Rightarrow \text{id} + T \Rightarrow \text{id} + \text{id}$, e a derivação mais à direita é $E \Rightarrow E + T \Rightarrow E + \text{id} \Rightarrow T + \text{id} \Rightarrow \text{id} + \text{id}$.

Existem duas abordagens principais de *parsing*: *top-down* e *bottom-up*. Na abordagem *top-down*, a análise parte do símbolo inicial da gramática e tenta derivar a cadeia de entrada por meio de derivações mais à esquerda. O analisador LL é um exemplo dessa abordagem: o nome indica que a entrada é lida da esquerda para a direita (*Left-to-right*) e que a derivação construída é a mais à esquerda (*Leftmost derivation*). Na variante LL(1), uma única unidade de entrada à frente (*lookahead*) basta para decidir qual produção aplicar (SINGH, 2009). Na abordagem *bottom-up*, a análise parte dos terminais da entrada e reduz a cadeia até o símbolo inicial da gramática, o que equivale a percorrer uma derivação mais à direita em ordem inversa. O analisador LR é o representante dessa abordagem: lê a entrada da esquerda para a direita (*Left-to-right*) e constrói uma derivação mais à direita em reverso (*Rightmost derivation*) (AHO et al., 2008). Os métodos da família LR diferem na forma como decidem entre empilhar um símbolo e reduzir por uma produção. O método SLR (*Simple LR*) é o mais simples: usa o conjunto FOLLOW do não terminal para decidir quando uma redução é permitida (SUNITHA, 2013). O método LALR (*Look-Ahead LR*) usa informação de *lookahead* mais precisa, obtida pela fusão de estados que possuem o mesmo núcleo, e é o método adotado por geradores como o *yacc* (AHO et al., 2008).

Ainda no *front-end*, a análise semântica recebe a árvore sintática e verifica se o programa possui significado coerente de acordo com as regras da linguagem. Uma construção pode ser sintaticamente válida e semanticamente inválida, como no caso de uma operação aritmética entre operandos de tipos incompatíveis (SINGH, 2009). A verificação de tipos é uma das funções centrais dessa fase, na qual se verifica se cada operador recebe operandos com tipos permitidos pela especificação da linguagem (AHO et al., 2008). Para isso, a análise semântica consulta a tabela de símbolos, introduzida anteriormente, nas verificações de declaração, compatibilidade de tipos e uso correto de variáveis (SUNITHA, 2013). O resultado é uma árvore sintática anotada, na qual cada nó contém informações de tipo associadas (SUNITHA, 2013), que segue para o *back-end*.

No *back-end*, a geração de código intermediário recebe a árvore sintática e a traduz em uma representação intermediária do programa, independente de qualquer arquitetura de máquina (SINGH, 2009). Essa representação funciona como uma interface entre as fases de análise e as fases de síntese do compilador, permitindo que otimizações sejam aplicadas de forma independente da linguagem-fonte e da máquina-alvo (AHO et al., 2008).

O otimizador de código recebe a representação intermediária e aplica transformações sobre ela com o objetivo de melhorar o desempenho do programa gerado, sem alterar seu comportamento, produzindo a representação intermediária otimizada. As otimizações podem visar a redução do tempo de execução, a redução do uso de memória ou ambos (SINGH, 2009).

Por fim, o gerador de código traduz a representação intermediária otimizada em código para a máquina-alvo, que pode ser código de máquina, código de montagem ou código para uma máquina virtual (SINGH, 2009).

2.2 Código de Três Endereços

A linguagem intermediária é utilizada em compiladores para traduzir o código-fonte para uma forma adequada a transformações de melhoria de código, antes da geração de código objeto ou de máquina (SINGH, 2009). Entre as representações intermediárias existentes estão o Código de Três Endereços (SUNITHA, 2013), o Grafo de Fluxo de Controle (AHO et al., 2008), a Representação Estática de Atribuição Única (CYTRON et al., 1991) e a Linguagem de Máquina Virtual (SMITH; NAIR, 2005).

O Código de Três Endereços é uma representação intermediária em que cada instrução envolve no máximo três operandos (SINGH, 2009). Essa representação lineariza a árvore de análise anotada em uma sequência de instruções simples, introduzindo variáveis temporárias para armazenar resultados intermediários de subexpressões (AHO et al., 2008). Os principais tipos de instrução do TAC são descritos a seguir.

Instruções de atribuição com operação possuem a forma `x = y op z`, em que `op` é um

operador aritmético, lógico ou relacional, `y` e `z` são operandos e `x` é o destino que recebe o resultado. Instruções de atribuição com operador unário possuem a forma `x = op y`, como no caso da negação lógica ou da inversão de sinal. Instruções de cópia possuem a forma `x = y`, em que o valor de `y` é transferido diretamente para `x`, sem operação intermediária (AHO et al., 2008).

Desvios incondicionais possuem a forma `goto L`, em que `L` é um rótulo que identifica a instrução de destino do salto. Desvios condicionais possuem a forma `if x op y goto L`, em que a execução é desviada para o rótulo `L` caso a condição `x op y` seja satisfeita, ou prossegue para a instrução seguinte caso contrário (AHO et al., 2008).

Para ilustrar a tradução de uma expressão para TAC, considere a expressão aritmética `a + b * c`. Como a multiplicação tem precedência sobre a adição, a subexpressão `b * c` é avaliada primeiro, e seu resultado é armazenado em um temporário:

```
t1 = b * c
t2 = a + t1
```

O TAC pode ser representado internamente em diferentes estruturas de dados. O formato *quadruple* utiliza registros com quatro campos: operador, argumento1, argumento2 e resultado. O formato *triple* simplifica essa estrutura para três campos, operador, argumento1 e argumento2, utilizando o índice da própria instrução como referência ao resultado em vez de um campo explícito (SINGH, 2009).

O uso do TAC permite *retargeting* mais simples, otimização de código independente de máquina e geração de código mais direta (SINGH, 2009). O *retargeting* é a capacidade de adaptar um compilador para gerar código para uma nova arquitetura de máquina reaproveitando o que já existe: como o *front-end* produz uma representação intermediária independente da máquina, basta escrever um novo gerador de código para a arquitetura desejada, sem alterar as fases de análise (AHO et al., 2008). Isso reduz o esforço de construção: com N linguagens-fonte e M arquiteturas-alvo, uma representação intermediária comum exige N *front-ends* e M *back-ends*, em vez de $N \times M$ compiladores completos (AHO et al., 2008). Algumas linguagens de alto nível geram apenas uma representação intermediária portátil, que é posteriormente interpretada ou compilada em diferentes plataformas, como ocorre em Java, cujo código-fonte é compilado para *bytecode* e executado pela *Java Virtual Machine* (JVM), e em C#, cujo código é compilado para a *Common Intermediate Language* (CIL) e executado pelo *Common Language Runtime* (CLR).

2.3 Máquinas Virtuais

Máquinas virtuais (do inglês *virtual machines*, VMs) têm origem teórica na Máquina de Turing Universal, proposta por Turing em 1936, que estabeleceu um modelo formal para o que é computável (SINGH, 2009). Uma VM reproduz um ambiente de computação real, permitindo

múltiplos ambientes em um único sistema físico (GOLDBERG, 1973). Popek e Goldberg (1974) descrevem os requisitos arquitetônicos de hardware para sistemas de VMs, com ênfase em eficiência, isolamento e equivalência de comportamento entre a VM e o sistema hospedeiro.

Smith e Nair (2005) classificam as VMs em duas categorias. As VMs de sistema emulam um sistema de hardware completo, incluindo processador, memória e dispositivos de entrada e saída, permitindo a execução de um sistema operacional inteiro sobre outro. As VMs de processo operam no nível de uma aplicação individual, executando código intermediário gerado por um compilador em um ambiente isolado, sem emular o hardware subjacente.

No contexto de linguagens de programação, as VMs de processo são o modelo adotado. A *Java Virtual Machine* (JVM) é um exemplo dessa categoria: o código-fonte Java é compilado para uma representação intermediária denominada *bytecode*, que é interpretado ou compilado em tempo de execução pela JVM, independentemente da plataforma de hardware (AHO et al., 2008). O *Common Language Runtime* (CLR) da plataforma .NET segue o mesmo princípio, executando código compilado para *Common Intermediate Language* (CIL). O interpretador CPython executa *bytecode* Python em uma VM baseada em pilha.

Em termos de arquitetura interna, as VMs de processo podem ser classificadas em dois modelos. As VMs baseadas em pilha utilizam uma pilha de operandos como estrutura central de armazenamento: os operandos são empilhados antes da operação, e o resultado substitui os operandos consumidos no topo da pilha. A JVM e o CPython seguem esse modelo. As VMs baseadas em registradores utilizam um conjunto de registradores virtuais para armazenar operandos e resultados, de forma análoga a um processador físico. A VM Lua e a Dalvik (utilizada em versões anteriores do Android) seguem esse modelo (SMITH; NAIR, 2005).

Para ilustrar a diferença entre os dois modelos, considere a operação `a + b`. Em uma VM baseada em pilha, a sequência de instruções empilha os dois operandos e aplica a operação:

```
push a
push b
add
```

Em uma VM baseada em registradores, a instrução especifica diretamente os registradores de origem e destino:

```
add r1, a, b
```

O modelo baseado em pilha tende a produzir instruções mais simples e em maior quantidade, enquanto o modelo baseado em registradores tende a produzir menos instruções, porém mais complexas (SMITH; NAIR, 2005).

Em termos de arquitetura geral, uma máquina abstrata inclui tipicamente um conjunto de instruções, um contador de programa que indica a instrução corrente, uma área de memória para armazenamento de dados e um mecanismo de execução que interpreta cada instrução e atualiza o estado da máquina (SINGH, 2009). Nisan e Schocken (2005) demonstram, no projeto Nand2Tetris, a construção de uma VM baseada em pilha que executa código intermediário, seguindo esse modelo de arquitetura.

2.4 Trabalhos Relacionados

Ferramentas que geram ou executam código intermediário com fins educacionais têm sido desenvolvidas em diferentes contextos.

O *Compiler Explorer*, também conhecido como Godbolt, é uma ferramenta web criada em 2012 que permite a um usuário escrever código-fonte em diversas linguagens e visualizar em tempo real a saída gerada por diferentes compiladores, incluindo código de montagem e representações intermediárias. A ferramenta é de código aberto e suporta múltiplos compiladores e linguagens (Compiler Explorer, 2025).

O PLY (*Python Lex-Yacc*) é uma implementação em Python das ferramentas tradicionais *lex* e *yacc*, utilizada para construção de analisadores léxicos e sintáticos, com base no algoritmo de *parsing* LALR(1). A ferramenta foi originalmente criada em 2001 para apoiar a disciplina de compiladores na Universidade de Chicago, sendo voltada a fins didáticos desde sua concepção (BEAZLEY, 2025).

O LLVM (*Low Level Virtual Machine*) é uma infraestrutura de compilador que utiliza uma representação intermediária (IR) própria, permitindo a criação e o aprimoramento de linguagens de programação (YEGULALP, 2023). A IR do LLVM funciona como uma camada portátil entre o código-fonte e o código de máquina, independente de arquitetura (YEGULALP, 2023). Ferramentas complementares, como o LLVM-FLOW, foram desenvolvidas para visualização interativa do Grafo de Fluxo de Controle gerado a partir da IR do LLVM, permitindo a comparação entre código antes e depois de otimizações (KC-ML2, 2023).

O CodeProber é uma ferramenta para desenvolvimento de compiladores e analisadores estáticos que expõe ao desenvolvedor as etapas intermediárias da compilação, permitindo a visualização do funcionamento interno do compilador durante o processamento de um programa. A ferramenta combina *liveness*, exploração de árvore sintática abstrata e apresentação de resultados de análise de programa sobre o código-fonte, e estava em uso ativo em contextos de pesquisa e ensino há mais de dois anos (ALAKÜLÄ; FORS; SÖDERBERG, 2025).

Boll (2023) apresenta a Celestial Suite, um conjunto de ferramentas desenvolvido na UNIOESTE para tradução e execução de código de três endereços. A suíte é composta por um compilador denominado Compass, implementado em Rust, que recebe código em uma

representação intermediária estendida (ETAC, *Extended Three-Address Code*) e o traduz para *assembly* MIPS, e por uma VM denominada Sextant, que executa o código MIPS gerado. O MIPS (*Microprocessor without Interlocked Pipeline Stages*) é uma arquitetura de processador do tipo RISC (*Reduced Instruction Set Computer*), caracterizada por um conjunto reduzido de instruções simples e de formato regular, o que a torna amplamente utilizada no ensino de arquitetura de computadores e de geração de código. A interface web Horizon integra os componentes, permitindo ao usuário inserir código intermediário, visualizar o *assembly* correspondente e acompanhar a execução na VM.

2.5 Diferenças em Relação aos Trabalhos Relacionados

As ferramentas descritas na Seção 2.4 cobrem etapas específicas do processo de compilação ou execução, sem abranger o ciclo completo. O *Compiler Explorer* expõe a saída de compiladores existentes, sem implementar as fases de análise (Compiler Explorer, 2025). O PLY é um gerador de analisadores léxicos e sintáticos, não contemplando análise semântica, geração de código intermediário ou execução (BEAZLEY, 2025). O LLVM e ferramentas associadas, como o LLVM-FLOW, atuam na camada de representação intermediária e otimização, sem cobrir a análise léxica, sintática ou semântica do código-fonte (YEGULALP, 2023). O CodeProber expõe estados internos do compilador durante análise sintática e semântica, mas não contempla geração de código intermediário ou execução (ALAKÜLÄ; FORS; SÖDERBERG, 2025).

A Celestial Suite traduz código intermediário para *assembly* MIPS e o executa em uma máquina virtual, cobrindo as etapas finais do pipeline de compilação, mas não implementa as fases de análise léxica, sintática ou semântica de uma linguagem-fonte (BOLL, 2023).

Esta pesquisa difere dos trabalhos relacionados por implementar, em um único sistema, todas as etapas do processo de compilação: análise léxica por AFD, análise sintática por dois métodos distintos (SLR e LL(1)), análise semântica com tabela de símbolos, geração de TAC e execução do código gerado por meio da Berti VM. Cada fase é implementada manualmente, sem uso de ferramentas geradoras automáticas, e expõe sua trajetória interna ao usuário, incluindo a sequência de estados do autômato léxico, os passos da pilha sintática e a verificação semântica regra a regra.

Esta pesquisa difere dos trabalhos relacionados por implementar, em um único sistema, todas as etapas do processo de compilação: análise léxica por AFD, análise sintática por dois métodos distintos (SLR e LL(1)), análise semântica com tabela de símbolos, geração de TAC e execução do código gerado por meio da Berti VM. Cada fase é implementada manualmente, sem uso de ferramentas geradoras automáticas, e expõe sua trajetória interna ao usuário, incluindo a sequência de estados do autômato léxico, os passos da pilha sintática e a verificação semântica regra a regra. Para situar essa diferença, a Tabela 1 compara as ferramentas apresentadas nesta seção com o presente trabalho segundo quatro critérios: as etapas do processo de compilação que

cada uma cobre, a linguagem-fonte aceita, se o código gerado é de fato executado e o nível de visualização do processo interno oferecido ao usuário.

Tabela 1 – Comparação entre ferramentas educacionais relacionadas a código intermediário

Ferramenta	**Etapas cobertas**	**Linguagem-fonte**	**Execução real**	**Visualização do processo interno**
Compiler Explorer	Compilação até código de montagem/IR	Múltiplas (C, C++, Rust, Go, etc.)	Não	Saída final do compilador
PLY	Análise léxica e sintática	Definida pelo usuário	Não	Tabela de *parsing* (*debug*)
LLVM / LLVM-FLOW	IR e otimização	Múltiplas (via *front-ends*)	Não (apenas geração de IR)	Grafo de Fluxo de Controle
CodeProber	Análise sintática e semântica	Definida pelo usuário (via Extendj)	Não	Estados intermediários do compilador, em tempo real
Celestial Suite	Tradução de TAC para MIPS e execução em VM	ETAC (linguagem intermediária própria)	Sim (VM Sextant)	Código MIPS gerado e estado da VM
Bertol (este trabalho)	Léxica, sintática, semântica, geração de TAC e execução	Bertol (linguagem própria)	Sim (Berti VM)	Trajetória completa de todas as fases

Conforme a Tabela 1, o Compiler Explorer, o PLY, o LLVM e o CodeProber não executam o código gerado, limitando-se à geração de código ou à análise. A Celestial Suite é a única, entre as ferramentas comparadas, que inclui execução, por meio da VM Sextant, mas parte de código intermediário já pronto: não cobre as fases de análise léxica, sintática e semântica. O Bertol é o único que reúne, em um mesmo sistema, todas as fases, a execução do código gerado pela Berti VM e a visualização da trajetória interna de cada etapa. Além disso, a Berti VM aceita um formato genérico de TAC, podendo executar o código intermediário de outras linguagens.

3 Metodologia

Este capítulo descreve a metodologia adotada no desenvolvimento deste trabalho. A Seção 3.1 retoma o problema abordado. A Seção 3.2 apresenta uma visão geral do que foi desenvolvido e da abordagem adotada. As Seções 3.3 a 3.6 detalham, fase a fase, como cada componente do compilador foi implementado. A Seção 3.7 descreve a Berti VM, incluindo o formato de TAC aceito, o modelo de dados e de memória, o ciclo de execução e a interface. A Seção 3.8 descreve o processo de validação adotado para o compilador e para a Berti VM.

3.1 Contexto e Problema

Conforme apresentado na Seção 2.1, o processo de compilação é composto por seis fases: análise léxica, análise sintática, análise semântica, geração de código intermediário, otimização de código e geração de código final. Na disciplina de Compiladores do curso de Ciência da Computação da UNIOESTE, campus Cascavel, todas as fases são abordadas teoricamente, porém a prática dos estudantes se encerra na geração de código intermediário, na forma de Código de Três Endereços (TAC). As etapas de otimização e geração de código final são estudadas apenas no plano teórico, e a execução do código gerado não é contemplada na disciplina.

Essa configuração resulta em uma lacuna na experiência do estudante: o código intermediário produzido ao final do trabalho prático não é executado, permanecendo como um artefato sem verificação de comportamento. Embora o estudante possa executar o código gerado de forma independente, isso exige ferramentas externas e conhecimento adicional, sem integração com o compilador desenvolvido na disciplina, o que dificulta verificar se o programa compilado produz o resultado esperado. Como mostrado na Seção 2.4, existem ferramentas que atendem parte dessa necessidade, porém nenhuma cobre o ciclo completo, da linguagem-fonte à execução, em um único sistema integrado com fins didáticos.

Diante disso, este trabalho propõe uma solução por meio da construção de três artefatos

integrados: a linguagem Bertol; um compilador para essa linguagem, que cobre as fases de análise léxica, sintática, semântica e geração de TAC; e a Berti VM, responsável por executar o código intermediário gerado, fechando o ciclo da gramática à execução. As seções seguintes descrevem como cada componente foi projetado e implementado.

3.2 Visão Geral

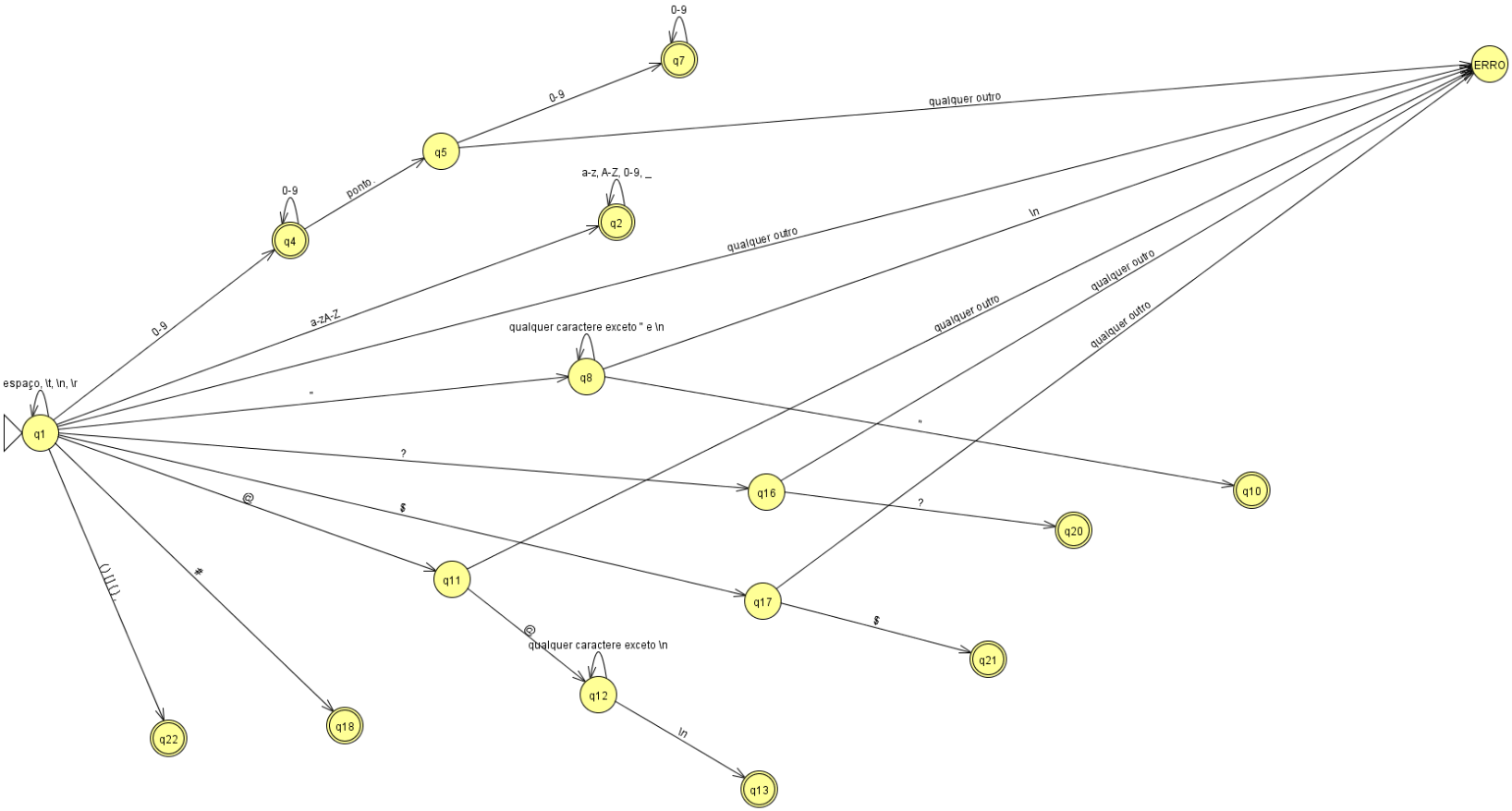
Este trabalho compreende o desenvolvimento de três artefatos integrados: a linguagem de programação Bertol, um compilador completo para essa linguagem e uma VM denominada Berti VM. O compilador é dividido em quatro fases: análise léxica, implementada por um Autômato Finito Determinístico (AFD); análise sintática, implementada por dois analisadores, pelo método SLR (*Simple LR*) e pelo método LL(1); análise semântica, com tabela de símbolos e verificação de tipos; e geração de Código de Três Endereços (TAC). A Berti VM interpreta e executa o TAC gerado pelo compilador, completando o ciclo da gramática à execução. Além do TAC da Bertol, ela aceita um formato genérico de TAC(GALANTE, 2026), com vetores, ponteiros e chamadas de função, de modo que pode executar o código intermediário de outras linguagens desenvolvidas em sala de aula.

A implementação foi realizada inteiramente em JavaScript, sem dependências externas, executando no navegador web. A escolha por uma implementação manual, em vez do uso de geradores de analisadores léxicos e sintáticos, decorre da decisão metodológica de priorizar a compreensão direta dos algoritmos envolvidos em cada fase (COOPER; TORCZON, 2012). Cada fase do compilador expõe ao usuário sua trajetória interna de execução, incluindo a sequência de estados percorridos, os passos da pilha sintática e a verificação semântica regra a regra, com suporte a dois modos de operação, com e sem recuperação de erros. A Berti VM segue a mesma abordagem didática: permite executar o TAC instrução por instrução, de forma contínua com velocidade ajustável ou de uma só vez, e exibe durante a execução o contador de programa, as variáveis em memória, a memória indexada de vetores e ponteiros, a pilha de chamadas e as operações de entrada e saída.

3.3 Análise Léxica

O analisador léxico foi implementado por meio de um Autômato Finito Determinístico (AFD) com 22 estados, responsável por converter o código-fonte em uma sequência de tokens. A Figura 2 apresenta o diagrama do AFD implementado.

Figura 2 – Autômato Finito Determinístico do analisador léxico da linguagem Bertol



Fonte: o autor

A Tabela 2 apresenta as categorias de tokens reconhecidas pelo AFD, com os padrões correspondentes descritos por expressões regulares.

Tabela 2 – Categorias de tokens da linguagem Bertol e suas expressões regulares

Categoria	**Expressão regular**	**Exemplos**
Identificador	`[a-zA-Z][a-zA-Z0-9_]*`	`limite`, `contador`
Número inteiro	`[0-9]+`	`0`, `42`, `100`
Número decimal	`[0-9]+\.[0-9]+`	`3.14`, `0.5`
Literal de texto	`"[^"]*"`	`"Olá"`, `"valor:"`
Operadores lógicos	`$$` \| `??` \| `XX`	`$$`, `??`, `XX`
Comentário	`@@.*`	`@@ comentário`
Fim de instrução	`#`	`#`
Delimitadores	`(` | `)` | `{` | `}` | `[` | `]` | `,`	`(`, `{`, `[`
Palavras reservadas	—	`SE`, `SENAO`, `ENQUANTO`, `PARA`, `PARE`, `VALE`, `RECEBE`, `MOSTRA`, `DE`, `ATE`, `PASSO`, `VERDADEIRO`, `FALSO`
Palavras reservadas (tipos)	—	`INTEIRO`, `DECIMAL`, `TEXTO`, `LOGICO`
Palavras reservadas (operadores)	—	`ADD`, `SUB`, `MULT`, `DIV`, `RESTO`, `MAIOR`, `MENOR`, `MAG`, `MEG`, `IGUAL`, `DIFERENTE`

A notação das expressões regulares da Tabela 2 é descrita a seguir:

a) `[a-z]`: classe de caracteres, que reconhece um único caractere dentre os do intervalo indicado; `[0-9]` reconhece qualquer dígito;

b) `[^"]`: classe negada, que reconhece qualquer caractere, exceto as aspas duplas;

c) `*`: zero ou mais repetições do elemento anterior;

d) `+`: uma ou mais repetições do elemento anterior;

e) `.`: qualquer caractere, exceto a quebra de linha; para reconhecer o ponto literal, usa-se `\.`;

f) `|`: alternativa, isto é, um ou outro padrão. Na linha dos delimitadores, a vírgula (`,`) não é um operador da notação: é um dos caracteres reconhecidos, usado como separador de itens, como em `MOSTRA["valor:", x]#`.

Os operadores lógicos da linguagem são `$$`, que representa a conjunção (E, ou *AND*), `??`, que representa a disjunção (OU, ou *OR*), e `XX`, que representa a negação (NÃO, ou *NOT*). Diferentemente dos dois primeiros, que são formados por símbolos, o `XX` é formado por letras e, por isso, é reconhecido pelo AFD como as palavras reservadas, descritas a seguir.

As palavras reservadas são reconhecidas inicialmente como identificadores pelo AFD. Após o reconhecimento do lexema completo, a função `obterToken` consulta uma tabela interna de palavras reservadas para determinar se o lexema deve ser classificado como palavra reservada ou como identificador.

O processamento ocorre caractere a caractere. O estado inicial do autômato é o estado `q1`, que distribui o fluxo de leitura de acordo com a categoria do caractere lido. Caracteres de espaçamento, como o espaço, a tabulação horizontal (`\t`), a quebra de linha (`\n`, do inglês *line feed*) e o retorno de carro (`\r`, do inglês *carriage return*), são descartados em `q1`, sem gerar token. Letras direcionam para o estado `q2`, responsável pelo reconhecimento de identificadores e palavras reservadas. Dígitos direcionam para o estado `q4`, que reconhece números inteiros. A partir de `q4`, a leitura de um ponto decimal direciona para o estado `q5`, que aguarda o primeiro dígito da parte fracionária, e em seguida para o estado `q6`, que reconhece os dígitos restantes da parte decimal. O caractere aspas duplas direciona para o estado `q8`, que acumula os caracteres do literal de texto até encontrar as aspas de fechamento. O caractere `@` direciona para o estado `q11`, que exige um segundo `@`: se ele vier, o autômato segue para o estado `q12`, que acumula o comentário de linha até a quebra de linha (`\n`); por isso o comentário exige dois caracteres `@`, e um `@` isolado é um erro léxico. Os caracteres `?` e `$` direcionam para os estados `q16` e `q17`, que reconhecem os operadores lógicos `??` e `$$`, compostos por dois caracteres idênticos consecutivos.

Os estados do AFD se dividem em dois grupos. Nos estados de aceitação, como `q2` (identificador), `q4` (inteiro), `q6` (decimal) e `q12` (comentário), o que foi lido até o momento já forma um lexema válido. Nos estados intermediários, como `q5`, `q8`, `q11`, `q16` e `q17`, o lexema ainda está incompleto: falta o dígito após o ponto (`q5`), as aspas de fechamento (`q8`) ou o segundo caractere do operador ou do comentário (`q11`, `q16` e `q17`).

A recuperação de erros léxicos segue uma estratégia de reinício no estado inicial. Ao detectar um erro, o analisador registra a mensagem com a linha e a coluna, descarta o lexema parcialmente reconhecido, retorna ao estado `q1` e prossegue a análise; no modo sem recuperação, a análise é interrompida no primeiro erro. O que diferencia os estados é o tratamento do caractere que causou o erro.

Nos estados `q5`, `q11`, `q16` e `q17`, o caractere é reprocessado em `q1`, sem avanço do cursor de leitura, pois pode ser o início de um novo token válido. Por exemplo, em `x VALE 3.#`, o autômato lê `3.` e chega a `q5`; o caractere seguinte, `#`, não é um dígito, então é registrado o erro *"Número mal formado '3.'"*. Em seguida, o `#` é reprocessado em `q1` e reconhecido normalmente como fim de instrução.

Já nos estados `q1` e `q8`, o cursor avança antes do retorno ao estado inicial, pois o caractere que causou o erro não pode iniciar um token. Em `q1`, esse caractere é um símbolo inválido: em `x VALE 5 & 3#`, o `&` gera o erro *"Caractere inesperado: '&'"* e é descartado. Em `q8`, é a quebra de linha que interrompe uma string não fechada, como em `MOSTRA["Olá#` seguido de uma nova linha, gerando o erro *"String não fechada"*; a quebra de linha é descartada.

Ao final do arquivo, pode restar um lexema ainda em construção. Nesse caso, o estado em que o autômato se encontra determina o resultado. Nos estados de aceitação, o lexema é válido e o token é emitido normalmente: se o código termina em `x VALE 10`, o lexema `10` está em `q4` e é emitido como número inteiro. Nos estados intermediários `q5`, `q8`, `q11`, `q16` e `q17`, o lexema está incompleto e é emitido como erro: se o código termina em `x VALE 3.`, o lexema `3.` está em `q5`, e se termina em `MOSTRA["abc`, o lexema está em `q8`, e ambos geram erro.

3.4 Análise Sintática

A análise sintática foi implementada por meio de dois parsers independentes, pelo método SLR (*Simple LR*), e pelo método LL(1). Ambos os parsers recebem como entrada a sequência de tokens produzida pela análise léxica e verificam se essa sequência está em conformidade com a gramática formal da linguagem Bertol.

3.4.1 Gramática Livre de Contexto

A linguagem Bertol é definida por uma Gramática Livre de Contexto (GLC) cujas produções são apresentadas a seguir. A produção vazia é representada por `epsilon`, e o símbolo `|` separa as diferentes derivações possíveis de um mesmo não terminal.

```
S'              -> lista_comandos
lista_comandos  -> comando lista_comandos | comando
comando         -> declaracao | atribuicao | entrada
                 | saida | condicional | laco_repeticao | break
declaracao      -> ID [ tipo ] #
atribuicao      -> ID [ tipo ] VALE expr #
                 | ID VALE expr #
entrada         -> RECEBE [ ID ] #
saida           -> MOSTRA [ lista_valores ] #
lista_valores   -> valor_saida | valor_saida , lista_valores
valor_saida     -> STRING | ID
condicional     -> SE ( expr ) { lista_comandos } opcional_senao
opcional_senao  -> SENAO { lista_comandos } | epsilon
laco_repeticao  -> laco_enquanto | laco_para
laco_enquanto   -> ENQUANTO ( expr ) { lista_comandos }
laco_para       -> PARA ID DE valor_for ATE expr
                   PASSO valor_for { lista_comandos }
valor_for       -> NUM_INT | NUM_REAL | ID
break           -> PARE #
expr            -> term expr_tail
```

```
expr_tail       -> binop term expr_tail | epsilon
binop           -> ?? | $$ | MENOR | MAIOR | IGUAL | MAG
                 | MEG | DIFERENTE | ADD | SUB | MULT
                 | DIV | RESTO
term            -> XX term | ( expr ) | valor
valor           -> NUM_INT | NUM_REAL | STRING | BOOLEAN | ID
tipo            -> INTEIRO | DECIMAL | TEXTO | LOGICO
```

Na notação da gramática, a diferença entre elementos minúsculos e maiúsculos é a seguinte:

a) os elementos escritos em minúsculas, como `comando`, `expr` e `tipo`, são símbolos não terminais, isto é, nomes de estruturas da linguagem que são definidas por outras produções da gramática;

b) os elementos escritos em maiúsculas, como `ID`, `VALE`, `SE` e `INTEIRO`, são símbolos terminais, isto é, os tokens produzidos pela análise léxica: as palavras reservadas e as categorias `ID` (identificador), `NUM_INT` (número inteiro), `NUM_REAL` (número decimal), `STRING` (literal de texto) e `BOOLEAN` (`VERDADEIRO` ou `FALSO`). Os símbolos de pontuação, como `#`, `(`, `)`, `[`, `]`, `{`, `}` e `,`, e os operadores lógicos `??` e `$$` também são terminais.

O símbolo `S'` é o símbolo inicial da gramática aumentada, utilizada pelo parser SLR. A seguir, a sintaxe de cada estrutura da linguagem é descrita com um exemplo. Todo comando simples termina com o símbolo `#`.

Declaração e atribuição. A declaração de uma variável informa o nome e o tipo entre colchetes, e os tipos disponíveis são `INTEIRO`, `DECIMAL`, `TEXTO` e `LOGICO`. A atribuição usa a palavra reservada `VALE` e pode ser feita junto com a declaração ou depois dela, em uma variável já declarada:

```
idade[INTEIRO]#
preco[DECIMAL] VALE 3.5#
nome[TEXTO] VALE "Ana"#
ativo[LOGICO] VALE VERDADEIRO#
idade VALE 20#
```

Expressões. Os operadores da linguagem são palavras reservadas: `ADD`, `SUB`, `MULT`, `DIV` e `RESTO` para operações aritméticas; `MAIOR`, `MENOR`, `MAG` (maior ou igual), `MEG` (menor ou igual), `IGUAL` e `DIFERENTE` para comparações; e os símbolos `$$` (E), `??` (OU) e `XX` (NÃO) para operações lógicas. Parênteses agrupam subexpressões:

```
soma VALE a ADD b#
resto VALE n RESTO 2#
valido VALE (x MAG 0) $$ (x MEG 10)#
```

Entrada e saída. O comando RECEBE lê um valor do usuário e o armazena em uma variável. O comando MOSTRA exibe uma lista de itens separados por vírgula, em que cada item é um literal de texto ou um identificador:

```
MOSTRA["Digite um numero: "]#
RECEBE[n]#
MOSTRA["Resultado: ", soma]#
```

Condicional. A estrutura SE executa o bloco entre chaves quando a condição é verdadeira, e a cláusula SENAO, opcional, executa o bloco alternativo:

```
SE (resto IGUAL 0) {
    MOSTRA["Par"]#
}
SENAO {
    MOSTRA["Impar"]#
}
```

Laços de repetição. O laço ENQUANTO repete o bloco enquanto a condição for verdadeira. O laço PARA percorre os valores de uma variável de controle, do valor inicial (DE) até o final (ATE), com o incremento indicado em PASSO:

```
ENQUANTO (n MAIOR 0) {
    n VALE n SUB 1#
}

PARA i DE 1 ATE 10 PASSO 1 {
    MOSTRA[i]#
}
```

Interrupção de laço. O comando PARE interrompe o laço mais interno em que está contido:

```
ENQUANTO (i MENOR n) {
    SE (resto IGUAL 0) {
        PARE#
```

```
    }
    i VALE i ADD 1#
}
```

Como exemplo completo, o programa a seguir lê um número e informa se ele é par ou ímpar:

```
n[INTEIRO]#
resto[INTEIRO]#
MOSTRA["Digite um numero: "]#
RECEBE[n]#
resto VALE n RESTO 2#
SE (resto IGUAL 0) {
    MOSTRA["Par"]#
}
SENAO {
    MOSTRA["Impar"]#
}
```

Essa gramática é utilizada diretamente pelo parser SLR. Para o parser LL(1), foram necessárias alterações estruturais, descritas na Subseção 3.4.3.

3.4.2 Parser SLR

O parser SLR opera por meio de uma pilha de estados e duas tabelas de controle: a tabela de ações, indexada por estado e terminal, e a tabela de transições (*goto*), indexada por estado e não-terminal. O autômato SLR construído para a gramática da linguagem Bertol possui 107 estados.

A cada passo, o estado no topo da pilha e o terminal atual são utilizados para consultar a tabela de ações, que indica uma das três operações possíveis. Na operação de deslocamento (*shift*), o estado indicado pela ação é empilhado e o cursor de leitura avança para o próximo token. Na operação de redução (*reduce*), a produção indicada pela ação é aplicada: um número de estados correspondente ao tamanho do lado direito da produção é desempilhado, e a tabela de transições é consultada com o estado resultante no topo da pilha e o não-terminal do lado esquerdo da produção, determinando o novo estado a ser empilhado. Na operação de aceitação (*acc*), a análise é concluída com sucesso.

Quando não há ação definida para o par estado-terminal atual, ocorre um erro sintático. A recuperação é realizada por meio de um conjunto de tokens de sincronização, composto pelos tokens `#`, `}`, `$`, `SE`, `ENQUANTO`, `PARA`, `RECEBE`, `MOSTRA`, `ID` e `PARE`. Ao detectar um erro, o

cursor de leitura avança até encontrar um token pertencente a esse conjunto, a pilha de estados é reiniciada para o estado inicial, e a análise prossegue. A recuperação é interrompida caso o número de erros acumulados atinja dez.

3.4.3 Parser LL(1)

O parser LL(1) é um parser *top-down* preditivo que utiliza uma pilha de símbolos e uma tabela de derivação indexada por não terminal e terminal. Nesse método, o parser decide qual produção aplicar olhando apenas o terminal atual da entrada. Por isso, a gramática não pode ter duas produções do mesmo não terminal que comecem com os mesmos símbolos, pois um único terminal à frente não bastaria para escolher entre elas. Produções nessa situação são resolvidas por fatoração à esquerda, técnica que consiste em colocar em uma única produção o prefixo comum e deixar em um novo não terminal apenas a parte em que as alternativas se diferenciam. Outro problema típico dos parsers *top-down* é a recursão à esquerda, que ocorre quando um não terminal deriva a si mesmo como primeiro símbolo, como em $A \rightarrow A\,\alpha$, e faz o parser entrar em laço infinito. A gramática da Bertol não possui recursão à esquerda, de modo que apenas a fatoração à esquerda foi necessária, em três produções da gramática original utilizada pelo parser SLR.

A primeira alteração é na produção de `lista_comandos`. As duas alternativas da produção original, `lista_comandos → comando lista_comandos | comando`, iniciam com `comando`. A produção foi substituída por duas, em que o prefixo comum aparece uma única vez e o não terminal `lista_comandos_cont` decide se há mais comandos ou se a lista termina:

```
lista_comandos      -> comando lista_comandos_cont
lista_comandos_cont -> comando lista_comandos_cont | epsilon
```

A segunda alteração é na produção de `comando`. Na gramática original, as três formas de comando que tratam de variáveis iniciam com o terminal `ID`, e duas delas iniciam com `ID [ tipo ]`. São elas:

a) declaração sem valor inicial, como em `x[INTEIRO]#`;

b) declaração com valor inicial, como em `x[INTEIRO] VALE 5#`, em que o tipo é informado uma única vez, porque o comando está declarando a variável;

c) atribuição a uma variável já declarada, como em `x VALE 5#`, em que o tipo não é repetido, pois já foi informado na declaração.

Para que o parser decida com um único terminal, as produções foram fatoradas em etapas: após o `ID`, o terminal seguinte, `[` ou `VALE`, define se o comando é uma declaração ou uma atribuição, e após o fechamento do tipo, o terminal seguinte, `#` ou `VALE`, define se a declaração tem ou não valor inicial. Para isso foram criados os não terminais intermediários `cmd_id`, `cmd_id_cont` e `cmd_tipo_cont`:

```
comando        -> cmd_id | entrada | saida | condicional
               | laco_repeticao | break
cmd_id         -> ID cmd_id_cont
cmd_id_cont    -> [ tipo ] cmd_tipo_cont | VALE expr #
cmd_tipo_cont  -> # | VALE expr #
```

A terceira alteração é na produção de `lista_valores`, cujas duas alternativas, `valor_saida` e `valor_saida , lista_valores`, iniciam com `valor_saida`. Aplicou-se o mesmo procedimento:

```
lista_valores      -> valor_saida lista_valores_cont
lista_valores_cont -> , lista_valores | e
```

A cada passo da análise, o símbolo no topo da pilha é comparado ao terminal atual. Se o topo da pilha for um terminal e corresponder ao terminal atual, ambos são consumidos. Se o topo da pilha for um não terminal, a tabela de derivação é consultada para determinar a produção a ser aplicada, substituindo o não terminal pelo lado direito da produção na pilha.

A recuperação de erros no parser LL(1) distingue dois casos, conforme o símbolo que está no topo da pilha. Nos exemplos a seguir, a pilha é mostrada com o topo à esquerda, e `$` indica o fim da entrada.

No primeiro caso, o símbolo no topo da pilha é um terminal que não corresponde ao terminal atual. O parser registra o erro, descarta o terminal atual, avançando o cursor de leitura uma posição, e mantém o terminal esperado no topo da pilha, repetindo a comparação com o token seguinte. Por exemplo, no comando `RECEBE[n]]#`, há um `]` a mais. Depois de consumir `RECEBE [ n ]`, o parser espera o terminal `#`, mas encontra `]`:

```
Pilha: # lista_comandos_cont $     Buffer: ] # $
Erro: esperado '#', encontrado ']'  -> descarta ']'
Pilha: # lista_comandos_cont $     Buffer: # $
```

Após descartar o `]` excedente, o `#` esperado aparece e a análise prossegue normalmente. Quando o terminal esperado nunca aparece, os tokens seguintes são descartados um a um, gerando um erro para cada, até o fim da entrada. É o que ocorre em `x[INTEIRO VALE 5#`, em que falta o `]`: os tokens `VALE`, `5` e `#` são descartados em sequência.

No segundo caso, o símbolo no topo da pilha é um não terminal para o qual a tabela não define produção para o terminal atual. O parser registra o erro e verifica se o terminal atual pertence ao conjunto de tokens de sincronização. Se pertencer, o não terminal é descartado da pilha, sem avanço do cursor, e a análise tenta continuar com o símbolo seguinte da pilha. Por exemplo, em `x VALE #`, falta a expressão após o `VALE`. O topo da pilha é o não terminal

`expr`, e o terminal # não inicia nenhuma expressão. Como # é um token de sincronização, `expr` é descartado, o # passa a ser comparado com o terminal # que está logo abaixo na pilha, e o comando é encerrado.

Se o terminal atual não pertencer ao conjunto de sincronização, o parser descarta o terminal atual, avançando o cursor uma posição, e mantém o não terminal no topo da pilha. Por exemplo, em `x VALE 5 6#`, após o `5`, o topo é o não terminal `expr_tail`, e o terminal `6` (`NUM_INT`) não é válido nessa posição nem é de sincronização. O `6` é descartado, e o terminal seguinte, #, permite que `expr_tail` derive a produção vazia.

O conjunto de tokens de sincronização do parser LL(1) é o mesmo utilizado pelo parser SLR, acrescido do token `SENAO`. Assim como no parser SLR, a análise é interrompida quando o número de erros acumulados atinge dez.

3.5 Análise Semântica

A análise semântica percorre a sequência de tokens validada pela análise sintática, aplicando um conjunto de 22 regras semânticas definidas para a linguagem Bertol. As informações sobre variáveis declaradas, incluindo nome, tipo, estado de inicialização e linha de declaração, são armazenadas em uma tabela de símbolos, consultada durante toda a análise.

3.5.1 Regras Semânticas

A análise semântica verifica, sobre a estrutura já validada pela análise sintática, se o programa respeita as regras de significado da linguagem, como declaração prévia de variáveis e compatibilidade de tipos. A Tabela 3 apresenta as 22 regras semânticas da linguagem Bertol, organizadas por categoria. Cada regra é identificada pelo código `RS-`*nn*, em que `RS` é a sigla de Regra Semântica e *nn* é o número sequencial da regra, de 01 a 22. Os mesmos códigos são usados pelo analisador semântico para registrar o resultado de cada verificação, de modo que uma mensagem de erro ou de sucesso exibida ao usuário pode ser associada diretamente à regra correspondente.

Tabela 3 – Regras semânticas da linguagem Bertol

Regra	**Categoria**	**Descrição**
RS-01	Declaração	Não é permitido declarar o mesmo identificador duas vezes
RS-02	Declaração	O tipo declarado deve ser válido (verificado na análise sintática)
RS-03	Declaração	Na declaração com atribuição (VALE), o tipo da expressão deve ser compatível com o tipo declarado
RS-04	Atribuição	A variável deve estar declarada antes de receber uma atribuição
RS-05	Atribuição	O tipo da expressão atribuída deve ser compatível com o tipo da variável
RS-06	Uso	A variável deve estar declarada antes de ser utilizada em uma expressão
RS-07	Uso	A variável deve estar inicializada antes de ser utilizada
RS-08	Operadores	Operadores aritméticos (ADD, SUB, MULT, DIV, RESTO) exigem operandos do tipo INTEIRO ou DECIMAL
RS-09	Operadores	Operadores relacionais numéricos (MAIOR, MENOR, MAG, MEG) exigem operandos do tipo INTEIRO ou DECIMAL
RS-10	Operadores	Os operadores IGUAL e DIFERENTE exigem operandos do mesmo tipo ou ambos numéricos
RS-11	Operadores	Os operadores lógicos `$$` (E) e `??` (OU) exigem operandos do tipo LOGICO
RS-12	Operadores	O operador lógico XX (NÃO) exige operando do tipo LOGICO
RS-13	Inferência	Operações aritméticas entre dois INTEIRO resultam em INTEIRO, operações com pelo menos um DECIMAL resultam em DECIMAL
RS-14	Inferência	Operações relacionais e lógicas resultam em LOGICO
RS-15	Controle	A condição do comando SE deve ser do tipo LOGICO
RS-16	Controle	A condição do comando ENQUANTO deve ser do tipo LOGICO
RS-17	Controle	A variável de controle do comando PARA deve ser do tipo INTEIRO
RS-18	Controle	Os valores DE e PASSO do comando PARA devem ser do tipo INTEIRO
RS-19	Controle	O valor ATE do comando PARA deve ser do tipo INTEIRO
RS-20	Controle	O comando PARE deve estar dentro de um laço
RS-21	Entrada/Saída	O identificador no comando RECEBE deve estar declarado
RS-22	Entrada/Saída	Os itens do comando MOSTRA devem ser literais de texto ou identificadores declarados

As categorias da Tabela 3 agrupam as regras conforme o aspecto do programa que verificam:

a) **Declaração** (RS-01 a RS-03): garantem que cada variável seja declarada uma única vez e com tipo válido, e que o valor inicial, quando existir, seja compatível com o tipo declarado;

b) **Atribuição** (RS-04 e RS-05): garantem que só se atribua valor a variáveis já declaradas e com tipo compatível. Por exemplo, atribuir o literal `"Ana"` a uma variável `INTEIRO` viola a RS-05;

c) **Uso** (RS-06 e RS-07): garantem que toda variável usada em uma expressão tenha sido declarada e inicializada;

d) **Operadores** (RS-08 a RS-12): garantem que cada operador receba operandos de tipos permitidos. Por exemplo, o operador `ADD` aplicado a um texto viola a RS-08;

e) **Inferência** (RS-13 e RS-14): definem o tipo do resultado das operações, necessário para as verificações das demais regras;

f) **Controle** (RS-15 a RS-20): garantem que as condições de `SE` e `ENQUANTO` sejam lógicas, que o laço `PARA` use valores inteiros e que o comando `PARE` apareça dentro de um laço;

g) **Entrada/Saída** (RS-21 e RS-22): garantem que os identificadores usados em `RECEBE` e `MOSTRA` estejam declarados.

3.5.2 Compatibilidade de Tipos

A verificação de tipos segue a tabela de compatibilidade apresentada na Tabela 4. A conversão implícita de INTEIRO para DECIMAL é permitida, por se tratar de uma operação sem perda de informação (*widening*). A conversão de DECIMAL para INTEIRO não é permitida, por acarretar perda da parte fracionária (*narrowing*), sendo reportada como erro semântico.

Tabela 4 – Tabela de compatibilidade de tipos da linguagem Bertol

Destino	**Origem**	**Compatível**
INTEIRO	INTEIRO	Sim
INTEIRO	DECIMAL	Não (*narrowing*)
INTEIRO	TEXTO	Não
INTEIRO	LOGICO	Não
DECIMAL	DECIMAL	Sim
DECIMAL	INTEIRO	Sim (*widening*)
DECIMAL	TEXTO	Não
DECIMAL	LOGICO	Não
TEXTO	TEXTO	Sim
TEXTO	outros	Não
LOGICO	LOGICO	Sim
LOGICO	outros	Não

3.5.3 Implementação

A análise semântica é implementada por meio de uma travessia linear da sequência de tokens. O analisador reconhece padrões de tokens correspondentes a cada construção da linguagem e aplica as regras semânticas associadas a cada padrão, conforme descrito na Tabela 5.

Tabela 5 – Padrões de tokens e regras semânticas associadas

Padrão de tokens	**Regras aplicadas**
`ID [ TIPO ] #`	RS-01
`ID [ TIPO ] VALE expr #`	RS-01, RS-03
`ID VALE expr #`	RS-04, RS-05
`RECEBE [ ID ] #`	RS-21
`MOSTRA [ ... ] #`	RS-22
`SE ( expr )`	RS-15
`ENQUANTO ( expr )`	RS-16
`PARA ID DE ... ATE ... PASSO ...`	RS-04, RS-17, RS-18, RS-19
`PARE #`	RS-20

A inferência de tipo de expressões é realizada pela função `inferirTipoExpr`, que opera por descida recursiva sobre os tokens de uma expressão. O processo segue cinco passos: remoção de parênteses externos quando toda a expressão está contida entre eles, tratamento do operador unário XX quando a expressão inicia com esse token, localização do operador binário de menor precedência fora de parênteses (precedência 1 para lógicos, 2 para relacionais e 3 para aritméticos), divisão da expressão em subexpressões esquerda e direita a partir desse operador, e aplicação recursiva do mesmo processo em cada subexpressão. O tipo resultante é determinado pelas regras RS-08 a RS-14, de acordo com o operador e os tipos dos operandos.

A verificação do comando PARE (RS-20) consiste em determinar se ele está dentro de algum laço. Para isso, o analisador mantém uma pilha de blocos abertos: ao encontrar um ENQUANTO ou um PARA, empilha um bloco de laço; ao encontrar um SE ou um SENAO, empilha um bloco condicional; e a cada chave de fechamento (`}`) desempilha o bloco mais recente. O comando PARE é aceito se existir pelo menos um bloco de laço na pilha e é rejeitado caso contrário. No exemplo a seguir, o primeiro PARE é aceito, pois está dentro do ENQUANTO, mesmo estando também dentro de um SE; o segundo é rejeitado, pois o laço já foi fechado:

```
ENQUANTO (i MENOR 5) {      pilha: [laço]
    SE (i IGUAL 2) {        pilha: [laço, SE]
        PARE#               aceito: há um laço na pilha
    }                       pilha: [laço]
    i VALE i ADD 1#
}                           pilha: []
PARE#                       erro RS-20: nenhum laço na pilha
```

3.5.4 Recuperação de Erros

A recuperação de erros na análise semântica opera por meio de ações paliativas, específicas para cada tipo de inconsistência detectada. Diferentemente da análise sintática, o analisador semântico não descarta trechos do código: ao encontrar um erro, ele o registra e toma uma decisão sobre a tabela de símbolos ou sobre o tipo da expressão, de modo que a análise prossiga até o final do código e reporte o maior número possível de erros em uma única execução. A seguir, descrevem-se as ações aplicadas em cada situação.

Variável não declarada em uma expressão (RS-06). O erro é registrado e o identificador é inserido ficticiamente na tabela de símbolos com o tipo `INTEIRO`. Assim, os usos seguintes da mesma variável não geram novamente o erro de variável não declarada. Por exemplo, em `soma VALE y ADD 1#` seguido de `soma VALE y ADD 2#`, com `y` não declarada, apenas o primeiro uso é reportado.

Limitação da suposição do tipo `INTEIRO`. Como o tipo real da variável é desconhecido, a suposição evita erros em cascata apenas quando a variável é usada em um contexto numérico. Quando é usada depois em outro contexto, ela pode gerar erros de tipo que não existiriam se a variável tivesse sido declarada. Por exemplo, após a falha de `y`, o comando `nome VALE y#`, com `nome` do tipo `TEXTO`, é reportado como tipo incompatível, e o comando `SE (y)` é reportado como condição que não é do tipo `LOGICO`. Esse é o custo da estratégia adotada: o primeiro erro, a falta da declaração, é sempre o relevante, e os erros de tipo que dependem dessa variável devem ser lidos como consequência dele.

Variável não declarada em `RECEBE` ou `MOSTRA` (RS-21 e RS-22). O erro é apenas registrado, e a tabela de símbolos não é alterada.

Condição de `SE` ou `ENQUANTO` que não é do tipo `LOGICO` (RS-15 e RS-16), ou variável de controle do `PARA` que não é do tipo `INTEIRO` (RS-17). O erro é registrado e a análise continua normalmente dentro do bloco, sem alterar a tabela de símbolos.

Operador aplicado a operandos de tipos incompatíveis (RS-08 a RS-12). O erro é registrado e um tipo de retorno é atribuído à expressão de forma forçada, o que permite que as expressões que dependem desse resultado continuem sendo avaliadas. O tipo forçado é `LOGICO` para os operadores relacionais e lógicos, e `INTEIRO` ou `DECIMAL` para os operadores aritméticos, conforme os tipos dos operandos.

Comando `PARE` fora de um laço (RS-20). O erro é apenas registrado, o comando é consumido e a análise continua.

No modo de operação sem recuperação de erros, a análise semântica é interrompida no primeiro erro detectado, e nenhuma das ações acima é aplicada.

3.6 Geração de Código de Três Endereços

A geração de código é a última fase do compilador, executada apenas quando a análise semântica é concluída sem erros. O gerador percorre a mesma sequência de tokens analisada nas fases anteriores, produzindo instruções no formato de Código de Três Endereços (TAC), nas quais cada instrução possui no máximo três operandos.

3.6.1 Estruturas Internas

O gerador utiliza três estruturas internas: um *array* de *strings* que acumula as instruções TAC geradas, dois contadores incrementais para nomeação de variáveis temporárias (`t1, t2, t3...`) e rótulos (`L1, L2, L3...`), e uma pilha de contextos de laços ativos, utilizada para o tratamento do comando PARE.

3.6.2 Geração de Expressões

A geração de código para expressões é realizada pela função `gerarExpr`, que recebe os tokens de uma expressão, emite as instruções TAC correspondentes e retorna o endereço (temporário ou variável) onde o resultado fica armazenado. O comportamento da função depende da estrutura da expressão, conforme descrito no Algoritmo 1.

1 expressão é um token único (variável ou constante) retorna o próprio lexema como endereço;
2 operador é aritmético ou lógico (`AND`, `OR`) esq ← gerarExpr(subexpressão esquerda);
3 dir ← gerarExpr(subexpressão direita);
4 t ← novoTemporario();
5 emitir `t = esq op dir`;
6 retorna t;
7 operador é relacional esq ← gerarExpr(subexpressão esquerda);
8 dir ← gerarExpr(subexpressão direita);
9 t ← novoTemporario();
10 Ltrue ← novoRotulo();
11 Lfim ← novoRotulo();
12 emitir `if esq op dir goto Ltrue`;
13 emitir `t = 0`;
14 emitir `goto Lfim`;
15 emitir `Ltrue:`;
16 emitir `t = 1`;
17 emitir `Lfim:`;
18 retorna t;
19 operador é unário (XX) operando ← gerarExpr(subexpressão);
20 t ← novoTemporario();
21 emitir `t = not operando`;
22 retorna t;

Algorithm 1: gerarExpr(exprToks)

O Algoritmo 1 é recursivo e sempre retorna um endereço: o nome de uma variável, uma constante ou um temporário que guarda o resultado parcial. Seus casos são os seguintes:

a) expressão de um único token: é o caso-base da recursão. Nenhuma instrução é emitida, e o próprio lexema é o endereço retornado; as constantes `VERDADEIRO` e `FALSO` são convertidas em `1` e `0`;

b) operador aritmético ou lógico binário: a função gera o código da subexpressão esquerda e da direita, obtém um endereço de cada uma, cria um temporário e emite uma única instrução que combina os dois endereços. Os operadores lógicos `$$` e `??` são emitidos como `and` e `or`;

c) operador relacional: o resultado da comparação é convertido em um valor lógico (0 ou 1) guardado em um temporário, como detalhado adiante;

d) operador unário `XX`: a função gera o código do operando e emite `t = not operando`.

A localização do operador principal da expressão segue a ordem de precedência da linguagem. Do menor para o maior grau de precedência, são: o operador lógico `??` (1), o

operador lógico `$$` (2), os operadores relacionais (3), os operadores aritméticos de adição e subtração, `ADD` e `SUB` (4), e os de multiplicação, divisão e resto, `MULT`, `DIV` e `RESTO` (5). A função localiza o operador de menor precedência fora de parênteses e, havendo empate, escolhe o último deles, o que torna os operadores associativos à esquerda. Em seguida, divide a expressão em subexpressões esquerda e direita a partir desse operador e aplica o processo recursivamente em cada uma. O operador de menor precedência é escolhido primeiro porque é o último a ser calculado: as subexpressões em torno dele precisam estar prontas antes. Assim, a expressão `2 MULT 3 ADD 4` é dividida no `ADD`, e o código gerado calcula primeiro a multiplicação:

```
t1 = 2 * 3
t2 = t1 + 4
```

Operadores relacionais requerem tratamento diferenciado porque o resultado precisa ser representado como um valor lógico (0 ou 1), e não como o resultado direto da comparação. Isso é necessário porque a instrução condicional do TAC, `if ... goto`, apenas desvia o fluxo e não produz valor, enquanto o resultado da comparação pode ser usado por outros operadores, como um `$$`. Para isso, é emitido um padrão de seis instruções que utiliza um desvio condicional para atribuir o valor correspondente a um temporário: se a comparação for verdadeira, o desvio leva ao trecho que atribui 1; caso contrário, o fluxo atribui 0 e salta esse trecho. Por exemplo, a expressão `x MAG a $$ x MEG b` gera:

```
if x >= a goto L1
t1 = 0
goto L2
L1:
t1 = 1
L2:
if x <= b goto L3
t2 = 0
goto L4
L3:
t2 = 1
L4:
t3 = t1 and t2
```

Os dois blocos de seis instruções calculam as comparações em `t1` e `t2`, e a última instrução combina os dois valores com o operador lógico. Os dois lados do operador lógico são sempre calculados, isto é, não há avaliação em curto-circuito.

3.6.3 Geração de Estruturas Condicionais

O Algoritmo 2 descreve a geração de código para o comando SE, com e sem a cláusula SENAO.

1 t ← gerarExpr(condição);
2 Ltrue ← novoRotulo();
3 Lfalse ← novoRotulo();
4 Lfim ← novoRotulo();
5 emitir `if t == 1 goto Ltrue`;
6 emitir `goto Lfalse`;
7 emitir `Ltrue:`;
8 gerar instruções do corpo ENTÃO;
9 emitir `goto Lfim`;
10 emitir `Lfalse:`;
11 existe cláusula SENAO gerar instruções do corpo SENAO;
12 emitir `Lfim:`;

Algorithm 2: Geração do comando SE

O algoritmo usa três rótulos: `Ltrue`, onde começa o corpo ENTÃO; `Lfalse`, que marca o início do bloco SENAO, ou o fim do comando quando ele não existe; e `Lfim`, o ponto de encontro dos dois caminhos. O código gerado avalia a condição, obtendo um temporário com valor 0 ou 1. Se o valor for 1, o fluxo salta para `Ltrue` e executa o corpo ENTÃO; caso contrário, salta para `Lfalse`. Ao terminar o corpo ENTÃO, o desvio `goto Lfim` é necessário para que o fluxo não continue para dentro do corpo SENAO. Sem SENAO, os rótulos `Lfalse` e `Lfim` ficam consecutivos. Por exemplo, o comando `SE (a MAIOR b) { MOSTRA["x"]# } SENAO { MOSTRA["y"]# }` gera:

```
if a > b goto L1
t1 = 0
goto L2
L1:
t1 = 1
L2:
if t1 == 1 goto L3
goto L4
L3:
print "x"
goto L5
L4:
print "y"
L5:
```

As cinco primeiras linhas calculam a condição `a MAIOR b` no temporário `t1`, conforme o padrão relacional da seção anterior. Em seguida, `L3` é o corpo ENTÃO, `L4` é o corpo SENAO e `L5` é o ponto de encontro.

Como o gerador percorre os tokens de forma linear, ao encontrar a chave que fecha o corpo ENTÃO ele ainda não sabe se existe uma cláusula SENAO. Por isso, nesse ponto ele consulta o token seguinte: se for SENAO, o contexto do comando permanece aberto e, ao encontrar a palavra-chave SENAO, o gerador emite `goto Lfim` e o rótulo `Lfalse`, passando do estado `then` para o estado `else`; caso contrário, o comando é finalizado imediatamente. Somente a chave que fecha o corpo SENAO emite o rótulo `Lfim`.

3.6.4 Geração de Estruturas de Repetição

O Algoritmo 3 descreve a geração de código para o comando ENQUANTO.

1 Linicio ← novoRotulo();
2 Ltrue ← novoRotulo();
3 Lfim ← novoRotulo();
4 empilhar contexto de laço com Lfim;
5 emitir `Linicio:`;
6 t ← gerarExpr(condição);
7 emitir `if t == 1 goto Ltrue`;
8 emitir `goto Lfim`;
9 emitir `Ltrue:`;
10 gerar instruções do corpo;
11 emitir `goto Linicio`;
12 emitir `Lfim:`;
13 desempilhar contexto de laço;

Algorithm 3: Geração do comando ENQUANTO

No Algoritmo 3, o rótulo `Linicio` é emitido antes do código da condição, de modo que a condição seja reavaliada a cada iteração. Se o valor for 1, o fluxo segue para o corpo, em `Ltrue`; caso contrário, salta para `Lfim` e sai do laço. Ao final do corpo, o desvio `goto Linicio` retorna ao início para uma nova avaliação. O contexto de laço empilhado guarda o rótulo `Lfim` do laço aberto mais internamente: é com ele que o comando PARE gera o desvio `goto Lfim` que interrompe o laço. O comando `ENQUANTO (n MAIOR 0) { n VALE n SUB 1# }` gera:

```
L1:
if n > 0 goto L4
t1 = 0
goto L5
L4:
```

```
t1 = 1
L5:
if t1 == 1 goto L2
goto L3
L2:
t2 = n - 1
n = t2
goto L1
L3:
```

O rótulo `L1` é o `Linicio`, as linhas até `L5` calculam a condição, `L2` é o corpo e `L3` é a saída do laço.

O Algoritmo 4 descreve a geração de código para o comando PARA.

1 emitir `varCtrl = valorInicial`;
2 Linicio ← novoRotulo();
3 Ltrue ← novoRotulo();
4 Lfim ← novoRotulo();
5 empilhar contexto de laço com Lfim;
6 emitir `Linicio:`;
7 tCond ← novoTemporario();
8 emitir `tCond = varCtrl <= valorFinal`;
9 emitir `if tCond == 1 goto Ltrue`;
10 emitir `goto Lfim`;
11 emitir `Ltrue:`;
12 gerar instruções do corpo;
13 tInc ← novoTemporario();
14 emitir `tInc = varCtrl + passo`;
15 emitir `varCtrl = tInc`;
16 emitir `goto Linicio`;
17 emitir `Lfim:`;
18 desempilhar contexto de laço;

Algorithm 4: Geração do comando PARA

O Algoritmo 4 reproduz a estrutura de um laço de contagem. A variável de controle é inicializada uma única vez, antes do rótulo `Linicio`. A cada iteração, a condição de permanência compara a variável de controle com o valor final, usando <=, de modo que o valor final é incluído no percurso. Se a condição for verdadeira, o corpo é executado; ao fim dele, a variável de controle é incrementada pelo valor do passo e o fluxo retorna a `Linicio`. Como a comparação usa <=, o laço pressupõe um passo positivo. O comando `PARA i DE 1 ATE 3 PASSO 1 { MOSTRA[i]# }` gera:

```
i = 1
L1:
t1 = i <= 3
if t1 == 1 goto L2
goto L3
L2:
print i
t2 = i + 1
i = t2
goto L1
L3:
```

Diferentemente do ENQUANTO, a condição de permanência no laço PARA não utiliza o padrão de seis instruções dos operadores relacionais: a comparação entre a variável de controle e o valor final é emitida diretamente em uma instrução, e o resultado, 0 ou 1, é armazenado em um temporário.

3.6.5 Tratamento do Comando PARE

O comando PARE é traduzido como um desvio direto para o rótulo de fim do laço mais interno em que o comando se encontra, conforme o Algoritmo 5.

1 percorrer pilha de contextos de laços do topo para a base;
2 Lfim ← rótulo de fim do laço mais próximo;
3 emitir `goto Lfim`;

Algorithm 5: Geração do comando PARE

A pilha de contextos de laços é empilhada a cada entrada em um comando ENQUANTO ou PARA e desempilhada ao final do bloco correspondente. Cada contexto armazena o rótulo `Lfim` do laço, permitindo que o comando PARE localize o destino correto do desvio.

3.6.6 Geração de Comandos de Entrada e Saída

O comando RECEBE é traduzido em uma instrução `read` seguida do identificador da variável. O comando MOSTRA é traduzido em uma ou mais instruções `print`, uma para cada item da lista de valores. Quando o item é um literal de texto, a instrução `print` recebe a *string* diretamente. Quando o item é um identificador, a instrução `print` recebe o nome da variável.

3.6.7 Geração de Declarações e Atribuições

Na declaração de variável com atribuição (`ID [ TIPO ] VALE expr #`), o gerador primeiro avalia a expressão por meio de `gerarExpr`, obtendo o endereço do resultado, e em

seguida emite uma instrução de atribuição `ID = endereço`. Quando o tipo da expressão é INTEIRO e o tipo declarado é DECIMAL, uma instrução de conversão `t = (float) endereço` é emitida antes da atribuição, correspondente à operação de *widening*.

Na declaração sem atribuição (`ID [ TIPO ] #`), nenhuma instrução TAC é emitida. Na atribuição simples (`ID VALE expr #`), o processo é o mesmo da declaração com atribuição.

3.6.8 Tradução de Bertol para TAC

Esta subseção apresenta três programas simples escritos em Bertol, o TAC que o compilador gera para cada um e a explicação de como cada linha do programa original foi traduzida. O objetivo é tornar visível, em casos concretos, o processo descrito nas subseções anteriores. Para acompanhar os exemplos, é útil ter em mente quatro convenções do gerador:

a) as declarações de variáveis não geram instruções, pois apenas registram o nome e o tipo na tabela de símbolos; uma declaração com valor inicial gera uma atribuição;

b) o resultado de uma operação é calculado primeiro em um temporário, nomeado em sequência (`t1`, `t2`, . . .), e depois copiado para a variável de destino;

c) os pontos de desvio do programa são marcados por rótulos, também nomeados em sequência (`L1`, `L2`, . . .), e a ordem numérica dos rótulos reflete a ordem em que o gerador os cria, não necessariamente a ordem em que aparecem no código;

d) uma comparação é convertida em um valor 0 ou 1, guardado em um temporário, por meio do padrão de seis instruções apresentado na geração de expressões, e esse valor é testado pelas estruturas de controle.

Exemplo 1: entrada, saída e atribuição. O programa lê dois números e exibe a soma. O código em Bertol é:

```
a[INTEIRO]#
b[INTEIRO]#
soma[INTEIRO]#
MOSTRA["Digite o primeiro numero: "]#
RECEBE[a]#
MOSTRA["Digite o segundo numero: "]#
RECEBE[b]#
soma VALE a ADD b#
MOSTRA["Soma: ", soma]#
```

O TAC gerado pelo compilador é:

```
print "Digite o primeiro numero: "
```

```
read a
print "Digite o segundo numero: "
read b
t1 = a + b
soma = t1
print "Soma: "
print soma
```

A Tabela 6 mostra qual trecho do TAC corresponde a cada linha do programa.

Tabela 6 – Tradução linha a linha do Exemplo 1

Código Bertol	**TAC gerado**	**Explicação**

`a[INTEIRO]#`
`b[INTEIRO]#`
`soma[INTEIRO]#`
	(nenhuma instrução)	A declaração apenas registra o nome e o tipo na tabela de símbolos; não há nada a executar.

`MOSTRA["Digite o primeiro numero: "]#` `RECEBE[a]#`	`print "Digite o primeiro numero: "` `read a`	`MOSTRA` vira `print` do texto, e `RECEBE` vira `read` da variável.
`MOSTRA["Digite o segundo numero: "]#` `RECEBE[b]#`	`print "Digite o segundo numero: "` `read b`	Mesma tradução, para a variável `b`.
`soma VALE a ADD b#`	`t1 = a + b` `soma = t1`	A soma é calculada no temporário `t1` e depois copiada para `soma`.
`MOSTRA["Soma: ", soma]#`	`print "Soma: "` `print soma`	Cada item da lista do `MOSTRA` gera um `print`.

Exemplo 2: condicional com SENAO. O programa informa se um número é par ou ímpar. O código em Bertol é:

```
n[INTEIRO]#
resto[INTEIRO]#
MOSTRA["Digite um numero: "]#
RECEBE[n]#
resto VALE n RESTO 2#
SE(resto IGUAL 0) {
    MOSTRA["Par"]#
}
SENAO {
    MOSTRA["Impar"]#
}
```

O TAC gerado pelo compilador é:

```
print "Digite um numero: "
read n
t1 = n % 2
resto = t1
if resto == 0 goto L1
t2 = 0
goto L2
L1:
t2 = 1
L2:
if t2 == 1 goto L3
goto L4
L3:
print "Par"
goto L5
L4:
print "Impar"
L5:
```

A Tabela 7 mostra a correspondência. Neste exemplo, o `SE` é traduzido em duas partes: primeiro o cálculo do valor 0 ou 1 da condição e depois a escolha do ramo a executar.

Tabela 7 – Tradução linha a linha do Exemplo 2

Código Bertol	**TAC gerado**	**Explicação**
`n[INTEIRO]#` `resto[INTEIRO]#`	(nenhuma instrução)	Declarações apenas registram as variáveis.
`MOSTRA["Digite um numero: "]#` `RECEBE[n]#`	`print "Digite um numero: "` `read n`	Saída do texto e leitura do valor.
`resto VALE n RESTO 2#`	`t1 = n % 2` `resto = t1`	O resto da divisão é calculado em `t1` e copiado para `resto`.

`SE(resto IGUAL 0) {` (cálculo da condição)	`if resto == 0 goto L1`
	`t2 = 0`
	`goto L2`
	`L1:`
	`t2 = 1`
	`L2:`
		Padrão de seis instruções: se `resto == 0`, o fluxo vai a `L1` e `t2` recebe 1; caso contrário, `t2` recebe 0 e o fluxo salta `L1`.

`SE(resto IGUAL 0) {` (escolha do ramo)	`if t2 == 1 goto L3`
	`goto L4`
	`L3:`
		Se a condição for verdadeira, o fluxo vai a `L3`, onde começa o corpo ENTÃO; caso contrário, vai a `L4`, onde começa o corpo SENAO.

`MOSTRA["Par"]#`	`print "Par"`	Corpo ENTÃO.
`}` `SENAO {`	`goto L5` `L4:`	Ao fim do corpo ENTÃO, o `goto L5` pula o corpo SENAO. O rótulo `L4` marca o início do corpo SENAO.
`MOSTRA["Impar"]#`	`print "Impar"`	Corpo SENAO.
`}` (fim do SENAO)	`L5:`	Ponto de encontro dos dois caminhos: a execução continua daqui, qualquer que tenha sido o ramo.

Exemplo 3: laço ENQUANTO. O programa realiza uma contagem regressiva. O código em Bertol é:

```
n[INTEIRO]#
MOSTRA["Digite um numero: "]#
RECEBE[n]#
ENQUANTO(n MAIOR 0) {
    MOSTRA[n]#
    n VALE n SUB 1#
}
MOSTRA["Fim!"]#
```

O TAC gerado pelo compilador é:

```
print "Digite um numero: "
read n
L1:
if n > 0 goto L4
t1 = 0
goto L5
L4:
t1 = 1
L5:
if t1 == 1 goto L2
goto L3
L2:
print n
t2 = n - 1
n = t2
goto L1
L3:
print "Fim!"
```

A Tabela 8 mostra a correspondência. Como no exemplo anterior, a condição é calculada em um temporário e depois testada, mas aqui o fluxo retorna ao início do laço ao final de cada iteração.

Tabela 8 – Tradução linha a linha do Exemplo 3

Código Bertol	**TAC gerado**	**Explicação**
`n[INTEIRO]#`	(nenhuma instrução)	Declaração apenas registra a variável.
`MOSTRA["Digite um numero: "]#` `RECEBE[n]#`	`print "Digite um numero: "` `read n`	Saída do texto e leitura do valor.

`ENQUANTO(n MAIOR 0)`
`{`
(início do laço)
	`L1:`	O rótulo `L1` marca o início do laço, e é para ele que o fluxo volta a cada iteração.

`ENQUANTO(n MAIOR 0)`
`{`
(cálculo da condição)
	`if n > 0 goto L4`
	`t1 = 0`
	`goto L5`
	`L4:`
	`t1 = 1`
	`L5:`
		Padrão de seis instruções: `t1` recebe 1 se `n > 0` e 0 caso contrário.

`ENQUANTO(n MAIOR 0)`
`{`
(teste da condição)
	`if t1 == 1 goto L2`
	`goto L3`
	`L2:`
		Se a condição for verdadeira, o fluxo vai a `L2`, o início do corpo; caso contrário, vai a `L3`, a saída do laço.

`MOSTRA[n]#`	`print n`	Primeira instrução do corpo.
`n VALE n SUB 1#`	`t2 = n - 1` `n = t2`	A subtração é calculada em `t2` e copiada para `n`.
`}` (fim do laço)	`goto L1` `L3:`	O `goto L1` volta ao início para reavaliar a condição. O rótulo `L3` marca a saída do laço.
`MOSTRA["Fim!"]#`	`print "Fim!"`	Instrução executada depois que o laço termina.

Os três exemplos mostram que o gerador traduz cada comando de forma local e previsível: os comandos simples geram instruções diretas, e as estruturas de controle geram um padrão fixo de rótulos e desvios, ao redor do TAC dos comandos que contêm. O TAC resultante é a entrada da Berti VM.

3.7 Berti VM

A Berti VM é a máquina virtual (VM) responsável por interpretar e executar o TAC gerado pelo compilador, etapa que completa o ciclo da gramática à execução proposto neste trabalho. Ela recebe como entrada o texto de um programa em Código de Três Endereços, executa suas instruções uma a uma e apresenta ao usuário, durante toda a execução, a instrução corrente, o conteúdo da memória, a pilha de chamadas e a saída do programa. Esta seção descreve as tecnologias empregadas, o formato de entrada aceito, o modelo de dados e de memória, o ciclo de execução, o tratamento de erros e a interface.

3.7.1 Tecnologias e Organização do Código

A Berti VM foi implementada em HTML, CSS e JavaScript puros, sem bibliotecas nem *frameworks*, e executa inteiramente no navegador. O código está dividido em três arquivos:

a) `index.html`: estrutura da página, com os painéis e os botões;

b) `style.css`: apresentação visual, que reutiliza a paleta de cores e a identidade visual do compilador Bertol;

c) `script.js`: toda a lógica da VM e a ligação com a interface.

O único recurso externo é a fonte tipográfica JetBrains Mono, carregada de um serviço de fontes, de uso apenas estético: sem conexão, o navegador usa uma fonte monoespaçada alternativa e a VM funciona normalmente. O arquivo `script.js` é executado dentro de uma função anônima, para que suas variáveis não interfiram no escopo global da página, e é organizado em seis blocos: o analisador das instruções (*parser*), o estado de execução, a execução passo a passo, o controle da execução, a renderização da interface e a ligação dos botões às funções.

3.7.2 Formato de Entrada: Especificação das Instruções

3.7.2.1 Regras Gerais

O programa de entrada é um texto simples, e a VM o interpreta segundo as regras a seguir:

a) cada linha contém um rótulo ou uma instrução; linhas em branco são ignoradas, e os espaços no início e no fim de cada linha são removidos;

b) um rótulo é um nome iniciado por letra ou pelo caractere `_`, seguido de letras, dígitos ou `_`, escrito sozinho em uma linha e terminado por dois-pontos, como em `L1:`. O rótulo identifica a posição da instrução seguinte e pode aparecer no fim do programa, caso em que um desvio para ele encerra a execução. Rótulos não são instruções e não ocupam posição na contagem de instruções;

c) as palavras-chave (`print`, `read`, `goto`, `if`, `ifFalse`, `param`, `call`, `return` e `not`) devem ser escritas exatamente assim, em minúsculas, com exceção de `ifFalse`, que tem o `F` maiúsculo;

d) em instruções com operador, o operando, o operador e o operando seguinte devem ser separados por espaços, como em `t1 = a + b`;

e) um operando pode ser um número inteiro (`42`, `-7`), um número decimal (`3.14`), uma constante lógica (`VERDADEIRO` ou `true`, `FALSO` ou `false`), um literal de texto entre aspas duplas (`"Ana"`) ou um nome, que designa uma variável, um temporário ou um vetor. Qualquer sequência de caracteres sem espaços pode ser usada como nome;

f) o texto de um literal é usado exatamente como foi escrito; sequências de escape, como \n, não são interpretadas, e o texto não pode conter aspas duplas;

g) uma variável que nunca recebeu valor e é lida pela primeira vez é criada com o valor 0 do tipo inteiro;

h) não existe sintaxe de comentário no TAC: uma linha que não corresponde a nenhum formato aceito é carregada normalmente, mas causa um erro quando a execução chega a ela.

3.7.2.2 Gramática

A gramática a seguir define formalmente o formato aceito. Os símbolos entre aspas são literais, os colchetes indicam elementos opcionais e as chaves indicam repetição (zero ou mais vezes):

```
programa       ::= { linha }
linha          ::= rotulo | instrucao
rotulo         ::= nome ":"
instrucao      ::= copia | binaria | neg_aritmetica | neg_logica
                 | cast | conversao | endereco | leitura_ptr
                 | escrita_ptr | leitura_vet | escrita_vet
                 | desvio | entrada | saida | param
                 | chamada | retorno
copia          ::= nome "=" operando
binaria        ::= nome "=" operando op operando
neg_aritmetica ::= nome "=" "-" nome
neg_logica     ::= nome "=" ( "not" | "!" ) operando
cast           ::= nome "=" "(" tipo ")" operando
conversao      ::= nome "=" funcao_conv "(" operando ")"
endereco       ::= nome "=" "&" nome
leitura_ptr    ::= nome "=" "*" nome
escrita_ptr    ::= "*" nome "=" operando
leitura_vet    ::= nome "=" nome "[" operando "]"
escrita_vet    ::= nome "[" operando "]" "=" operando
desvio         ::= "goto" nome
                 | "if" condicao "goto" nome
                 | "ifFalse" condicao "goto" nome
condicao       ::= operando | operando op operando
entrada        ::= "read" nome
saida          ::= "print" operando | "print" texto
param          ::= "param" operando
```

```
chamada        ::= [ nome "=" ] "call" nome "," inteiro
retorno        ::= "return" [ operando ]
op             ::= "+" | "-" | "*" | "/" | "%"
                 | ">" | "<" | ">=" | "<=" | "==" | "!="
                 | "and" | "&&" | "or" | "||"
operando       ::= inteiro | decimal | texto | nome
                 | "VERDADEIRO" | "true" | "FALSO" | "false"
```

Cada produção da gramática corresponde a um formato de instrução reconhecido pelo analisador da VM. Uma linha é, portanto, um rótulo ou uma instrução, e cada alternativa de `instrucao` é descrita, com o seu efeito, nas Tabelas 9 a 12. Os símbolos `nome`, `inteiro`, `decimal`, `texto`, `tipo` e `funcao_conv` não foram expandidos por serem léxicos: `nome` é qualquer sequência de caracteres sem espaços, `tipo` é `int`, `float`, `double` ou `char`, e `funcao_conv` é uma das funções da Tabela 14.

3.7.2.3 Instruções e Seus Efeitos

As Tabelas 9 a 12 descrevem o efeito de cada formato de instrução. Os formatos `x = y` e `x = y op z` são os únicos usados pelo código gerado para expressões e atribuições da Bertol, e os demais ampliam a VM para o formato genérico.

Tabela 9 – Instruções de atribuição e de operação

Formato	**Efeito**
`x = y`	Copia o valor e o tipo do operando `y` para `x`.
`x = y op z`	Aplica o operador `op` aos operandos `y` e `z` e grava o resultado em `x`. Os operadores estão descritos na Tabela 13.
`x = -y`	Negação aritmética: grava em `x` o valor de `y` com o sinal trocado, mantendo o tipo. O operando deve ser um nome; um número negativo escrito diretamente, como `-5`, é tratado como constante.
`x = not y` `x = !y`	Negação lógica: grava em `x` o valor lógico 1 se `y` valer 0 e 0 caso contrário.
`x = (tipo) y`	Conversão de tipo (*cast*). Os tipos aceitos são `float` e `double`, que geram um valor decimal, `int`, que trunca o valor para inteiro, e `char`, que gera um caractere.
`x = conv(y)`	Conversão nomeada, em que `conv` é uma das funções da Tabela 14.

A Tabela 9 reúne as instruções que calculam um valor e o gravam em uma variável. Na primeira coluna, `x` é sempre o destino, e `y` e `z` são operandos, que podem ser variáveis, temporários ou constantes. As duas primeiras linhas, a cópia e a operação binária, são as que o compilador Bertol efetivamente emite para atribuições e expressões. As demais (negação, *cast* e

conversão nomeada) existem para aceitar o TAC de outras linguagens, como o das linguagens com tipagem estática, em que o compilador insere conversões explícitas entre tipos numéricos.

Tabela 10 – Instruções de memória indexada: vetores e ponteiros

Formato	**Efeito**
`x = a[i]`	Lê o elemento de posição `i` do vetor `a` e o grava em `x`. A posição é somada ao endereço-base do vetor; uma posição nunca escrita vale 0.
`a[i] = y`	Grava o valor de `y` na posição `i` do vetor `a`. O vetor é criado na primeira vez em que é usado.
`p = &x`	Grava em `p` o endereço da variável `x`, com o tipo ponteiro.
`x = *p`	Lê o valor armazenado no endereço contido em `p` e o grava em `x`. Um endereço nunca escrito vale 0.
`*p = y`	Grava o valor de `y` no endereço contido em `p`.

A Tabela 10 mostra as instruções que acessam a memória indexada, descrita em detalhe na Seção 3.7.4. As duas primeiras linhas acessam um vetor por nome e posição, e as três últimas acessam a memória por endereço, usando ponteiros: `&x` obtém o endereço de uma variável, `*p` no lado direito lê o conteúdo apontado e `*p` no lado esquerdo escreve nele. Nenhuma dessas instruções é gerada pela Bertol, pois a linguagem não possui vetores nem ponteiros. Elas existem para que a VM execute o TAC de linguagens que os possuem, como a linguagem C.

Tabela 11 – Instruções de desvio

Formato	**Efeito**
`goto L`	Desvio incondicional: a execução continua na instrução que segue o rótulo `L`.
`if y op z goto L`	Desvia para `L` se a comparação ou operação entre `y` e `z` for verdadeira (resultado diferente de 0); caso contrário, segue para a instrução seguinte.
`if y goto L`	Desvia para `L` se o valor de `y` for diferente de 0.
`ifFalse y op z goto L`	Desvia para `L` se a comparação ou operação for falsa (resultado 0).
`ifFalse y goto L`	Desvia para `L` se o valor de `y` for 0.

A Tabela 11 lista as instruções que alteram o fluxo de execução, que é sequencial por padrão. Em todas elas, `L` é um rótulo definido em alguma linha do programa. Os desvios condicionais avaliam a condição e desviam se ela for verdadeira, no caso de `if`, ou falsa, no caso de `ifFalse`; se o desvio não for tomado, a execução prossegue na instrução seguinte. O compilador Bertol utiliza apenas `goto` e a forma `if y op z goto L`, e as formas com `ifFalse` e com condição de um só operando fazem parte do formato genérico.

Tabela 12 – Instruções de entrada, saída e funções

Formato	**Efeito**
`read x`	Pausa a execução e solicita um valor ao usuário, que é gravado em `x`.
`print y`	Exibe o valor do operando `y` em uma linha da saída.
`print "texto"`	Exibe o texto em uma linha da saída.
`param y`	Empilha o valor de `y` na pilha de parâmetros.
`call f, n`	Chama a função cujo rótulo é `f`, informando que `n` parâmetros foram empilhados. O resultado, se existir, é descartado.
`x = call f, n`	Igual à anterior, mas o valor retornado pela função é gravado em `x`.
`return` `return y`	Encerra a função e retorna ao ponto da chamada, opcionalmente com o valor de `y`.

A Tabela 12 reúne as instruções de interação com o usuário e de chamada de funções. As instruções `read` e `print` correspondem aos comandos `RECEBE` e `MOSTRA` da Bertol e são detalhadas na Seção 3.7.7. As instruções `param`, `call` e `return` implementam as chamadas de função e são detalhadas na Seção 3.7.5. O número `n` de `call f, n` deve coincidir com a quantidade de instruções `param` executadas imediatamente antes da chamada.

Tabela 13 – Operadores binários aceitos

Operador	**Significado**	**Tipo do resultado**
+, -, *	Soma, subtração e multiplicação	Decimal se algum operando for decimal; inteiro nos demais casos
/	Divisão. Entre dois inteiros, o resultado é truncado em direção a zero	Decimal se algum operando for decimal; inteiro nos demais casos
%	Resto da divisão, com o sinal do dividendo, como na linguagem C	Decimal se algum operando for decimal; inteiro nos demais casos
>, <, >=, <=	Comparações de ordem	Lógico (1 ou 0)
==, !=	Igualdade e diferença, que comparam apenas os valores	Lógico (1 ou 0)
and, &&	Conjunção: resultado 1 se os dois operandos forem diferentes de 0	Lógico (1 ou 0)
or, ||	Disjunção: resultado 1 se algum operando for diferente de 0	Lógico (1 ou 0)

A Tabela 13 define os operadores que podem aparecer em `x = y op z` e nas condições de `if` e `ifFalse`. A terceira coluna mostra que o tipo do resultado depende dos operandos nas operações aritméticas e é sempre lógico nas comparações e nas operações booleanas. Dois pontos merecem atenção: a divisão entre dois inteiros descarta a parte fracionária (`7 / 2` resulta em 3, e

`7.0 / 2` resulta em 3.5), e o sinal do resto acompanha o do dividendo (`-7 % 3` resulta em −1). Os operadores `and` e `or` aceitam também as grafias `&&` e `||` do formato genérico, e as operações da Bertol equivalentes, `??` e `$$`, já chegam ao TAC traduzidas para um desses operadores.

Tabela 14 – Conversões nomeadas aceitas

Função	**Efeito**
`inttofloat`, `inttodouble`, `floattodouble`, `doubletofloat`, `chartofloat`	O valor resultante é do tipo decimal.
`floattoint`, `doubletoint`, `chartoint`	O valor é truncado em direção a zero e o resultado é do tipo inteiro.
`inttochar`	O valor numérico é reclassificado como caractere.

A Tabela 14 agrupa as conversões nomeadas do formato genérico por tipo de resultado. Como a VM unifica `float` e `double` em um único tipo decimal (Seção 3.7.3), as conversões entre esses dois tipos não alteram o valor, e servem apenas para que o TAC de origem seja aceito sem modificação. As conversões para inteiro truncam o valor em direção a zero (3.9 resulta em 3 e −3.9 resulta em −3), e não o arredondam. O compilador Bertol emprega a forma *cast*, `t = (float) x`, descrita na Tabela 9.

3.7.3 Modelo de Dados

A Berti VM é dinamicamente tipada: cada valor armazenado em memória é um par formado pelo valor e pelo seu tipo, e o tipo de uma variável é o tipo do último valor que ela recebeu. Essa escolha é necessária porque o TAC não carrega as declarações de tipo do programa original. A Tabela 15 apresenta os tipos internos da VM e sua correspondência com os tipos da Bertol e com os tipos do formato genérico.

Tabela 15 – Tipos internos da Berti VM

Tipo interno	**Origem**	**Representação**	**Exibição na saída**
inteiro	`INTEIRO`; `int`	Número de ponto flutuante de 64 bits sem parte fracionária	Número sem casas decimais, como `42`
decimal	`DECIMAL`; `float` e `double`, unificados	Número de ponto flutuante de 64 bits	Número com casas decimais; se o valor for inteiro, exibe `.0`, como `4.0`
logico	`LOGICO`	Número 1 ou 0	`VERDADEIRO` ou `FALSO`
texto	`TEXTO`	Sequência de caracteres	O próprio texto
caractere	`char`	Valor numérico reclassificado	O valor numérico
ponteiro	Resultado de `&x` ou de um nome de vetor	Número inteiro que representa um endereço	O endereço precedido de `@`, como `@1000`

A Tabela 15 relaciona, para cada tipo interno, de onde ele vem, como é representado e como aparece na saída. A segunda coluna mostra que os tipos `INTEIRO`, `DECIMAL`, `LOGICO` e `TEXTO` da Bertol têm correspondência direta, e que o formato genérico entra com `int`, `float`, `double` e `char`. A terceira coluna mostra que inteiros e decimais usam a mesma representação numérica do JavaScript, e a diferença entre eles está no rótulo de tipo que acompanha o valor, o que também explica a exibição da quarta coluna: um decimal de valor 4 é mostrado como `4.0`, e um inteiro de valor 4 como `4`.

As regras de tipo seguem os pontos abaixo:

a) uma operação aritmética entre um inteiro e um decimal produz um decimal, ou seja, a promoção do inteiro para decimal é automática;

b) a cópia `x = y` preserva o tipo de `y`, e não há conversão automática no sentido oposto: a conversão de decimal para inteiro exige uma conversão explícita;

c) um valor lido pela instrução `read` tem o tipo inferido do texto digitado: uma sequência de dígitos, com sinal opcional, é inteiro; dígitos com um ponto decimal e ao menos um dígito de cada lado, como `2.5`, é decimal; qualquer outra entrada é texto.

3.7.4 Modelo de Memória

A memória da VM é composta por quatro estruturas:

a) a **memória de variáveis**, que associa o nome de cada variável ou temporário ao seu valor, com o tipo;

b) a **memória indexada**, que associa um endereço numérico a um valor e é usada por vetores e por variáveis cujo endereço foi tomado;

c) a tabela de **endereços-base**, que associa a cada vetor ou variável endereçável o endereço onde ela começa;

d) o conjunto dos **nomes de vetores**, que identifica quais nomes devem ser tratados como vetores.

O funcionamento segue as regras a seguir:

a) os endereços são alocados a partir de 1000, e cada novo vetor ou variável endereçável reserva um bloco de 100000 endereços. Assim, vetores diferentes nunca se sobrepõem. O primeiro vetor usa a partir de `@1000`, o seguinte a partir de `@101000`, e assim por diante;

b) na primeira instrução que usa um nome como vetor (`a[i]` em leitura ou escrita), a VM aloca o bloco do vetor. O elemento de posição `i` fica no endereço-base somado a `i`;

c) quando a instrução `p = &x` é executada pela primeira vez para uma variável `x`, a VM move `x` da memória de variáveis para a memória indexada, em um novo endereço. Daí em diante, qualquer uso de `x` por nome lê e escreve nesse endereço, o que mantém `x` e `*p` sempre consistentes;

d) o nome de um vetor usado sozinho como operando, como em `p = a`, vale o endereço-base do vetor, com o tipo ponteiro. Isso permite aritmética de ponteiros: depois de `q = p + 1`, a instrução `z = *q` lê o elemento seguinte do vetor. O resultado da soma é um inteiro, mas o seu valor continua sendo um endereço válido;

e) as posições dos vetores são contadas em unidades de elemento, e não em bytes. Se o TAC já calcula o deslocamento manualmente, como em `t1 = i * 4`, esse valor é somado ao endereço-base sem alteração, e os elementos ficam espaçados de quatro em quatro endereços;

f) a VM não verifica limites: qualquer posição é aceita, inclusive negativa ou maior que o bloco do vetor, caso em que o acesso pode invadir o bloco de outro vetor.

3.7.5 Funções, Pilha de Parâmetros e Pilha de Chamadas

Uma função, no TAC aceito pela VM, é um trecho de código iniciado por um rótulo e encerrado por uma instrução `return`. O corpo da função fica no texto do programa, e o código principal deve desviá-lo com um `goto`, como no exemplo da Seção 3.7.11, pois, caso contrário, a execução seguiria para dentro da função ao chegar nela sem uma chamada. A chamada de função envolve duas estruturas:

a) a **pilha de parâmetros**, na qual cada instrução `param y` empilha o valor de `y` no momento em que é executada;

b) a **pilha de chamadas**, na qual cada chamada empilha um quadro com a posição da instrução que segue a chamada, o ponto de retorno, e o nome da variável que receberá o valor retornado, quando a chamada tem destino.

A execução de `call f, n` ou `x = call f, n` verifica se o rótulo `f` existe, retira da pilha de parâmetros os `n` valores mais recentes, empilha o quadro na pilha de chamadas e desvia para o rótulo `f`. A instrução `return y` retira o quadro do topo da pilha de chamadas, grava o valor de `y` na variável de destino, se houver, e retoma a execução no ponto de retorno. Um `return` executado com a pilha de chamadas vazia encerra o programa. A cada chamada e a cada retorno, a VM escreve uma linha de destaque na saída, como `→ chamando soma()` e `← retorno: 9`.

O documento de referência(GALANTE, 2026) não especifica como o corpo de uma função acessa os parâmetros recebidos. Por isso, a VM adotou a convenção mais simples: todas as variáveis são globais e compartilhadas, e os valores retirados da pilha de parâmetros na chamada são descartados, sem associação a nomes. A função acessa os valores pelos nomes das variáveis que o código principal já usa, como `a` e `b` no exemplo da Seção 3.7.11. O número `n` informado na chamada serve apenas para decidir quantos valores retirar da pilha de parâmetros. A consequência dessa escolha é que não há escopo local por chamada: duas chamadas aninhadas ou recursivas que usem os mesmos nomes de variável sobrescrevem os valores uma da outra. A pilha de chamadas em si suporta qualquer profundidade e retorna corretamente a cada nível, e o problema se restringe ao compartilhamento das variáveis.

3.7.6 Ciclo de Execução

3.7.6.1 Carga do Programa

Ao acionar o botão *Carregar programa*, a VM limpa a saída, lê o texto de entrada e o converte em uma lista de instruções, associando cada instrução ao formato correspondente da gramática, e constrói a tabela de rótulos, que associa cada rótulo ao índice da instrução seguinte. A carga não valida o programa: uma linha que não corresponde a nenhum formato é aceita e só causa erro quando a execução chega a ela, da mesma forma que um rótulo inexistente só causa erro quando um desvio para ele é executado. Em seguida, a VM é reiniciada: a memória e as pilhas são esvaziadas, o contador de programa vai para zero e o estado passa a *pronta*.

3.7.6.2 Execução de uma Instrução

O estado de execução inclui o contador de programa (PC), que é o índice da instrução corrente, contado a partir de zero e ignorando os rótulos. A execução avança pela repetição do seguinte ciclo, que equivale a um passo:

a) se a VM estiver encerrada ou aguardando a entrada de um valor, nenhuma ação é realizada;

b) se o PC for igual ou maior que o número de instruções, o programa é considerado terminado;

c) o contador de passos é incrementado, e, se ultrapassar 200000, a execução é abortada com um erro, o que protege o navegador contra laços infinitos;

d) a instrução do PC é identificada pelo seu tipo, os operandos são resolvidos e a operação é realizada;

e) o PC é atualizado: nas instruções sequenciais, passa para a instrução seguinte; em um desvio tomado, passa para o índice do rótulo de destino; em uma chamada, passa para o rótulo da função; em um retorno, passa para o ponto de retorno guardado no quadro;

f) se ocorrer qualquer erro durante a operação, a execução é interrompida e a mensagem é exibida;

g) a interface é atualizada, com a instrução corrente, a memória, a memória indexada, a pilha de chamadas e a saída;

h) se o PC passou da última instrução, o programa termina.

Um programa termina em quatro situações: o PC ultrapassa a última instrução, o que inclui o desvio para um rótulo colocado no fim do programa; um `return` é executado com a pilha de chamadas vazia; ocorre um erro; ou o limite de 200000 instruções é atingido. Ao terminar, a VM escreve a mensagem `- fim do programa -` na saída.

3.7.7 Entrada e Saída

Ao executar a instrução `read x`, a VM não avança o PC: ela pausa a execução, mostra abaixo da saída um campo de texto identificado por `read x →` e muda o estado para *aguardando entrada*. O usuário digita o valor e o confirma com a tecla *Enter* ou com o botão *Enviar*. Uma entrada vazia é ignorada. A VM infere o tipo do valor conforme a Seção 3.7.3, grava o valor na variável, registra na saída uma linha como `read x = 5`, avança o PC e retoma a execução conforme o modo em uso. Como a VM não conhece as declarações de tipo da linguagem de origem, o usuário deve digitar um ponto decimal para que um valor seja tratado como decimal: o valor `5` é lido como inteiro mesmo que a variável original fosse decimal.

A instrução `print` escreve uma linha na saída para cada operando, formatado conforme o seu tipo (Tabela 15). Uma instrução `print "texto"` escreve o texto como está. O operando de `print` deve ser simples: um nome, um número ou um texto. Como o TAC gerado pela Bertol emite um `print` para cada item de uma lista `MOSTRA`, um comando com vários itens produz várias linhas de saída, uma por item.

3.7.8 Modos de Execução e Controles

A execução é controlada pelos botões do painel de execução, descritos a seguir:

a) *Carregar programa*: carrega o texto de entrada, como descrito na Seção 3.7.6;

b) *Step*: executa uma única instrução e deixa a VM pausada, de modo que o usuário acompanha cada mudança de estado;

c) *Run*: executa as instruções continuamente, com um intervalo entre elas definido pelo controle deslizante de velocidade, em que o intervalo, em milissegundos, é 601 menos o valor do controle, que varia de 1 a 600; o valor inicial 200 corresponde a 401 milissegundos. A interface é atualizada a cada instrução;

d) *Instantâneo*: executa o programa inteiro de uma só vez, sem intervalo entre as instruções e sem atualizar a interface durante a execução, o que torna possível rodar laços longos rapidamente. A interface é atualizada ao final, ou quando a execução pausa em um `read`;

e) *Pausar*: interrompe o modo *Run*, e a execução pode continuar a partir da mesma instrução com um novo acionamento de *Run*;

f) *Reiniciar*: limpa a saída, a memória e as pilhas e volta o PC a zero, mantendo o programa carregado.

Nos modos *Run* e *Instantâneo*, a execução pausa em cada `read` e é retomada automaticamente, no mesmo modo, depois que o valor é informado. Os botões *Step*, *Run* e *Instantâneo* ficam desabilitados quando o programa terminou, quando a VM aguarda uma entrada ou quando o modo *Run* está em andamento, e o botão *Pausar* só fica habilitado durante o modo *Run*. O estado da VM é indicado em uma etiqueta no canto superior: *ociosa* (sem programa), *pronta*, *executando*, *executando (instantâneo)*, *aguardando entrada*, *pausada*, *finalizada* e *erro*.

3.7.9 Tratamento de Erros

A VM não tenta se recuperar de erros de execução: ao detectar um, interrompe o programa, escreve em vermelho na saída a mensagem `Erro:` seguida da descrição, muda o estado para *erro* e aguarda que o usuário reinicie a execução. A Tabela 16 lista os erros detectados.

Tabela 16 – Erros detectados pela Berti VM

Situação	**Quando é detectada**	**Mensagem**
Desvio para um rótulo que não existe	Ao executar `goto`, `if` ou `ifFalse` com o desvio tomado	`rótulo indefinido: L`
Chamada de uma função cujo rótulo não existe	Ao executar `call`	`função/rótulo indefinido: f`
Linha que não corresponde a nenhum formato	Ao executar a linha	`instrução não reconhecida: <linha>`
Mais de 200000 instruções executadas	Durante a execução	`limite de 200000 instruções excedido (possível laço infinito)`

A Tabela 16 apresenta, para cada erro, a situação que o causa, o momento em que ele é detectado e a mensagem exibida ao usuário. Todos são erros de execução, e não de carga: um desvio para um rótulo inexistente, por exemplo, só é percebido se o desvio for de fato tomado. O último caso não indica necessariamente um defeito do programa, e sim a proteção contra laços infinitos, e um programa legítimo que precise de mais de 200000 instruções também seria interrompido.

Há situações que a VM não trata como erro, por serem responsabilidade do compilador ou por decisão de projeto: a divisão por zero produz o valor `Infinity` (e o resto por zero, `NaN`), uma variável nunca atribuída vale 0 e os acessos a vetores não têm verificação de limites.

3.7.10 Interface

A interface é dividida em dois grupos de painéis, numerados na ordem de uso:

a) **Painel 1, Entrada TAC**: campo de texto para o programa, contador de linhas e os botões *Carregar programa*, *Exemplo Bertol* e *Exemplo vetor / ponteiro / função*, que preenchem o campo com programas prontos;

b) **Painel 2, Execução**: lista das instruções do programa, em que cada instrução é numerada com o seu índice, os rótulos são exibidos sem número e a instrução corrente é destacada e mantida visível; indicador do PC; os botões de controle da seção anterior e o controle deslizante de velocidade;

c) **Painel 3, Memória**: tabela com o nome, o valor e o tipo de cada variável, em que as variáveis com endereço exibem o seu endereço (como `@101000`); lista da memória indexada, em que cada endereço é exibido com o valor, o tipo e, quando pertence a um vetor, o nome e a posição (como `v[1]`); e lista da pilha de chamadas e dos parâmetros pendentes;

d) **Painel 4, Saída**: as linhas escritas por `print`, as mensagens de leitura, de chamada e retorno de função e de erro, o botão *limpar* e o campo de entrada de valores do `read`.

A página também tem um botão *!*, no mesmo padrão do compilador, que abre um guia com o passo a passo de uso e os formatos de instrução aceitos. O *layout* segue a mesma paleta de cores do compilador Bertol, em tema escuro, de modo que as duas ferramentas formam um conjunto visualmente coerente.

3.7.11 Execução Passo a Passo

Os exemplos a seguir mostram o estado da VM depois de cada instrução, executadas em modo *Step*. O primeiro mostra uma chamada de função, com as pilhas de parâmetros e de chamadas:

```
a = 4
b = 5
param a
param b
r = call soma, 2
print r
goto Lfim
soma:
s = a + b
return s
Lfim:
```

A Tabela 17 mostra a execução. As instruções são numeradas de 0 a 8, o rótulo `soma` aponta para a instrução 7 e o rótulo `Lfim` aponta para a posição 9, logo após a última instrução.

Tabela 17 – Execução passo a passo do programa com chamada de função

Passo	**PC**	**Instrução**	**Efeito**
1	0	`a = 4`	A variável `a` é criada com o valor 4.
2	1	`b = 5`	A variável `b` é criada com o valor 5.
3	2	`param a`	O valor 4 é empilhado. Parâmetros pendentes: `[4]`.
4	3	`param b`	O valor 5 é empilhado. Parâmetros pendentes: `[4, 5]`.
5	4	`r = call soma, 2`	Os dois parâmetros são retirados da pilha e descartados. Um quadro com ponto de retorno 5 e destino `r` é empilhado, e a pilha de chamadas passa a ter 1 quadro. O PC passa a 7, a posição do rótulo `soma`. Saída: `→ chamando soma()`.
6	7	`s = a + b`	A variável `s` é criada com o valor 9.
7	8	`return s`	O quadro é retirado da pilha, a variável `r` recebe 9 e o PC volta a 5. Saída: `← retorno: 9`.
8	5	`print r`	Saída: `9`.
9	6	`goto Lfim`	O PC passa a 9, depois da última instrução. O programa termina. Saída: `- fim do programa -`.

A Tabela 17 deve ser lida linha a linha, na ordem em que as instruções são executadas, que não é a ordem do texto. Os passos 1 a 4 preparam a chamada: as variáveis recebem os valores e a pilha de parâmetros cresce até conter 4 e 5. No passo 5, a chamada consome os parâmetros, registra no quadro o ponto de retorno (a instrução 5, `print r`) e desvia para a função, e é por isso que o PC salta de 4 para 7. Os passos 6 e 7 executam o corpo da função, e o `return` do passo 7 faz o caminho inverso, devolvendo o PC a 5 e gravando o resultado em `r`. O `goto Lfim` do passo 9 evita que a execução entre no corpo da função pela segunda vez, o que a faria executar `return` com a pilha de chamadas vazia, e encerra o programa.

O segundo exemplo mostra a memória indexada, com um vetor e um ponteiro:

```
v[0] = 7
v[1] = 14
i = 1
x = v[i]
y = 3
p = &y
*p = 9
print y
```

A Tabela 18 mostra a execução.

Tabela 18 – Execução passo a passo do programa com vetor e ponteiro

Passo	**PC**	**Instrução**	**Efeito**
1	0	`v[0] = 7`	O vetor `v` é criado com endereço-base `@1000`. A memória indexada passa a ter `@1000` (`v[0]`) com o valor 7.
2	1	`v[1] = 14`	O endereço `@1001` (`v[1]`) recebe 14.
3	2	`i = 1`	A variável `i` é criada com o valor 1.
4	3	`x = v[i]`	O endereço é 1000 + 1 = 1001, e `x` recebe 14.
5	4	`y = 3`	A variável `y` é criada com o valor 3.
6	5	`p = &y`	A variável `y` é movida para a memória indexada, no endereço `@101000`, o próximo bloco livre. A variável `p` recebe o ponteiro `@101000`, e `y` passa a ser exibida com esse endereço.
7	6	`*p = 9`	O valor 9 é gravado no endereço `@101000`, e `y` passa a valer 9.
8	7	`print y`	Saída: `9`. O programa termina.

A Tabela 18 mostra os dois usos da memória indexada. Nos passos 1 a 4, o vetor `v` é preenchido e lido: o endereço de cada elemento é o endereço-base mais a posição, e a leitura `x = v[i]` usa o valor atual de `i` para calcular o endereço. Nos passos 5 a 8, a variável `y` passa a ter endereço quando `p = &y` é executada, e a escrita `*p = 9`, feita pelo ponteiro, altera a própria variável `y`, como comprova o `print y` do último passo, que mostra 9 e não 3. Isso evidencia que `y` e `*p` designam o mesmo local de memória.

3.8 Validação

A validação do sistema foi dividida em duas partes: a do compilador, que verifica as quatro fases implementadas, e a da Berti VM, que verifica a execução do TAC. Em ambas, o procedimento é o mesmo: cada caso de teste tem um resultado esperado, definido previamente a partir da especificação da linguagem ou do formato de TAC, e o resultado obtido é comparado a ele. Os resultados dos testes são apresentados no Capítulo 4.

3.8.1 Validação do Compilador

A validação do compilador foi realizada por meio de casos de teste escritos na linguagem Bertol, cobrindo as construções suportadas pela linguagem, incluindo declarações de variáveis dos quatro tipos primitivos, expressões aritméticas, relacionais e lógicas, estruturas condicionais e de repetição, o comando `PARE` e os comandos de entrada e saída. Cada caso de teste verifica a saída produzida em cada uma das quatro fases implementadas, léxica, sintática, semântica e geração de TAC. Foram definidos dois conjuntos de casos:

a) 14 programas válidos, que devem ser aceitos em todas as fases, sem nenhum erro, e para os quais o compilador deve gerar o TAC;

b) 11 programas com um erro intencional cada, sendo dois erros léxicos, três sintáticos e seis semânticos, que verificam o funcionamento dos mecanismos de recuperação de erros: o erro deve ser detectado na fase correspondente à sua natureza, a mensagem deve indicar a posição do erro e as fases seguintes devem prosseguir.

Cada programa foi analisado com a recuperação de erros ativada nas três fases de análise, e, nos programas válidos, os dois métodos sintáticos, SLR e LL(1), foram aplicados.

3.8.2 Validação da Berti VM

A validação da Berti VM teve dois objetivos: verificar que ela executa corretamente o TAC gerado pelo compilador e verificar que ela executa as instruções do formato genérico que a Bertol não emite, como vetores, ponteiros e funções. Cada objetivo foi atendido por um conjunto de testes próprio.

3.8.2.1 Execução do TAC Gerado pelo Compilador

O primeiro conjunto é formado pelos 10 primeiros programas válidos usados na validação do compilador, escritos em Bertol. O procedimento aplicado a cada programa foi:

a) compilar o programa com o compilador Bertol, verificando que as quatro fases terminaram sem erros;

b) copiar o TAC gerado, sem nenhuma alteração, para o painel de entrada da Berti VM e carregá-lo;

c) executar o programa na VM, informando as entradas necessárias quando há comandos `RECEBE`;

d) comparar a saída produzida pela VM com a saída esperada do programa, determinada previamente a partir do enunciado do programa e da semântica da linguagem Bertol;

e) repetir a execução com diferentes conjuntos de valores de entrada, incluindo valores limite, como zero e números negativos, o que resultou em 41 execuções no total.

Esse conjunto valida o ciclo completo do trabalho, da gramática à execução, e a compatibilidade entre o código gerado pelo compilador e o formato aceito pela VM. Ele exercita as instruções `x = y`, `x = y op z`, `if ... goto`, `goto`, `read` e `print`, ou seja, as instruções emitidas pelo gerador nesses programas, que têm expressões aritméticas e lógicas, estruturas condicionais e laços.

3.8.2.2 Execução de Instruções do Formato Genérico

Como a Bertol não possui vetores, ponteiros nem funções, o primeiro conjunto não exercita os recursos da VM destinados ao TAC de outras linguagens. O segundo conjunto,

formado por 5 programas escritos diretamente em TAC, foi criado para esse fim e cobre a memória indexada e as chamadas de função com passagem de parâmetros:

a) escrita e leitura de posições de vetores, com posição constante e com posição calculada por uma variável;

b) ponteiros, com a obtenção do endereço de uma variável (`p = &x`), a leitura pelo ponteiro (`x = *p`) e a escrita pelo ponteiro (`*p = y`);

c) chamada de função com passagem de parâmetros e retorno de valor (`param`, `call` e `return`);

d) chamadas de função em sequência e aninhadas;

e) uso conjunto de vetores e funções em um mesmo programa.

Para cada programa, a saída esperada foi calculada manualmente antes da execução, e o estado da memória foi acompanhado no modo *Step*, com o mesmo tipo de acompanhamento mostrado nas Tabelas 17 e 18.

3.8.2.3 Critérios de Aceitação

Um teste foi considerado aprovado quando todas as condições abaixo foram atendidas:

a) a VM terminou a execução sem exibir mensagem de erro;

b) a saída produzida foi idêntica à saída esperada, linha a linha;

c) os modos *Step*, *Run* e *Instantâneo* produziram a mesma saída para o mesmo programa e as mesmas entradas.

A última condição é relevante porque os três modos usam caminhos de código diferentes para controlar a execução, e uma divergência entre eles indicaria um defeito na interface, e não na semântica das instruções. Por isso, cada execução, tanto dos programas gerados pelo compilador quanto dos programas em TAC, foi repetida nos três modos.

4

Resultados

Este capítulo apresenta os resultados obtidos no desenvolvimento deste trabalho. A Seção 4.1 descreve o estado de implementação do compilador e da Berti VM. A Seção 4.2 apresenta um exemplo completo, ilustrando a transformação de um programa escrito em Bertol em Código de Três Endereços e a execução desse código na Berti VM. A Seção 4.3 descreve os resultados da validação por meio de casos de teste.

4.1 Estado de Implementação

As quatro fases do compilador, análise léxica, análise sintática, análise semântica e geração de Código de Três Endereços, encontram-se completamente implementadas. A análise léxica reconhece a totalidade dos tokens definidos na especificação da linguagem Bertol por meio do Autômato Finito Determinístico descrito na Seção 3.3. A análise sintática está implementada nos dois métodos propostos, SLR e LL(1), ambos capazes de processar a gramática completa da linguagem. A análise semântica aplica a totalidade das regras semânticas definidas para a linguagem, incluindo a verificação de tipos, a checagem de declaração de identificadores e a validação do uso de estruturas de controle. A geração de código produz instruções de Código de Três Endereços para todas as construções suportadas pela linguagem, incluindo expressões aritméticas, lógicas e relacionais, comandos de entrada e saída, estruturas condicionais e estruturas de repetição.

A Berti VM, responsável pela execução do TAC gerado, também se encontra completamente implementada, conforme descrito na Seção 3.7. Ela executa todas as instruções emitidas pelo gerador de código, além das instruções do formato genérico de TAC que a linguagem Bertol não emite, como vetores, ponteiros e funções. A VM oferece os modos de execução passo a passo, contínuo e instantâneo, e exibe durante a execução a instrução corrente, as variáveis, a memória indexada, as pilhas de chamadas e de parâmetros e a saída do programa. Com ela, o

ciclo proposto neste trabalho, da gramática à execução, está completo: um programa escrito em Bertol é analisado, traduzido para TAC e executado, e cada etapa pode ser acompanhada pelo usuário.

4.2 Exemplo de Execução

Para ilustrar o funcionamento do compilador e da Berti VM, considere o programa apresentado no Código 1, escrito em Bertol, que lê um valor inteiro e, caso esse valor seja maior que zero, exibe a sequência de números de zero até o valor informado.

Código 1 – Programa exemplo em Bertol

```
1  limite[INTEIRO] VALE 0#
2  contador[INTEIRO]#
3  MOSTRA["Informe um valor: "]#
4  RECEBE[limite]#
5  SE(limite MAIOR 0) {
6      PARA contador DE 0 ATE limite PASSO 1 {
7          MOSTRA[contador]#
8      }
9  }
10 SENAO {
11     MOSTRA["O limite deve ser maior que 0."]#
12 }
```

A análise léxica converte o programa em uma sequência de tokens, identificando os identificadores `limite` e `contador`, a palavra reservada `INTEIRO`, o operador de atribuição `VALE`, as palavras reservadas `MOSTRA`, `RECEBE`, `SE`, `SENAO` e `PARA`, o operador relacional `MAIOR`, os delimitadores de bloco e os literais numéricos e de texto presentes no programa. A análise sintática confirma que essa sequência está em conformidade com a gramática da linguagem, tanto pelo parser SLR quanto pelo parser LL(1). A análise semântica verifica que as variáveis `limite` e `contador` são declaradas antes de seu uso, que a condição do comando SE é do tipo LOGICO, resultante da aplicação do operador relacional MAIOR sobre dois operandos do tipo INTEIRO, e que a variável de controle `contador` do comando PARA é do tipo INTEIRO.

A geração de código produz a saída apresentada no Código 2.

As três primeiras instruções correspondem à inicialização da variável `limite`, à exibição da mensagem de solicitação e à leitura do valor informado pelo usuário. As instruções seguintes, até o rótulo `L2`, correspondem à avaliação da condição relacional do comando SE, conforme o padrão descrito na Seção 3.6, armazenando em `t1` o valor 1 caso `limite` seja maior que zero, ou 0 caso contrário. A partir do rótulo `L3`, é gerado o corpo correspondente à cláusula então do comando SE, que contém o comando PARA. A inicialização da variável de controle `contador`, o rótulo de início do laço `L6`, a avaliação da condição de permanência no laço, representada pelo temporário `t2`, o corpo do laço e a instrução de incremento da variável de controle, armazenada

Código 2 – Código de Três Endereços gerado a partir do Código 1

```
limite = 0
print "Informe um valor: "
read limite
if limite > 0 goto L1
t1 = 0
goto L2
L1:
t1 = 1
L2:
if t1 == 1 goto L3
goto L4
L3:
contador = 0
L6:
t2 = contador <= limite
if t2 == 1 goto L7
goto L8
L7:
print contador
t3 = contador + 1
contador = t3
goto L6
L8:
goto L5
L4:
print "O limite deve ser maior que 0."
L5:
```

no temporário `t3`, seguem o padrão descrito para o comando PARA. Ao final do laço, no rótulo `L8`, a instrução `goto L5` desvia a execução para o final da estrutura condicional, evitando a execução do bloco correspondente à cláusula SENAO, gerado a partir do rótulo `L4`. O rótulo `L5` está no final do programa e, portanto, o desvio para ele encerra a execução.

O TAC do Código 2 foi carregado na Berti VM sem nenhuma alteração. O programa tem 19 instruções, pois os rótulos não são contados (Seção 3.7). A Tabela 19 apresenta a execução para três valores de entrada.

Tabela 19 – Execução do Código 2 na Berti VM

Entrada	**Caminho percorrido**	**Saída do programa**
3	`t1` recebe 1 e o laço `PARA` é executado com `contador` de 0 a 3	`Informe um valor:` `0`, `1`, `2`, `3`, cada valor em uma linha
1	`t1` recebe 1 e o laço `PARA` é executado com `contador` de 0 a 1	`Informe um valor:` `0`, `1`, cada valor em uma linha
0 ou −1	`t1` recebe 0 e a execução desvia para `L4`, o bloco do `SENAO`	`Informe um valor:` `O limite deve ser maior que 0.`

A Tabela 19 mostra que a VM percorre os dois ramos do comando SE. Com entradas

positivas, a condição `limite > 0` é verdadeira, `t1` recebe 1 e a execução entra no laço, que imprime todos os valores de 0 até o limite, inclusive, pois a condição de permanência no laço é `contador <= limite`. Com entradas menores ou iguais a zero, `t1` recebe 0, o desvio `goto L4` é tomado e apenas a mensagem do bloco SENAO é impressa, sem que o laço seja executado. Em ambos os casos, o `goto L5` ou o fim das instruções leva o programa a terminar, e a VM exibe a mensagem de fim de programa. O comportamento observado é o mesmo que o programa original em Bertol especifica, o que mostra que cada etapa do compilador preservou a semântica do programa.

4.3 Validação

Esta seção apresenta os casos de teste definidos na Seção 3.8 e os resultados obtidos. A validação do compilador utilizou 14 programas válidos e 11 programas com erros intencionais. A validação da Berti VM utilizou 10 programas escritos em Bertol, executados com 41 conjuntos de entrada diferentes, além de 5 programas escritos diretamente em TAC para os recursos de memória indexada e de chamada de funções. Todos os programas executados na VM foram executados nos três modos, *Step*, *Run* e Instantâneo.

4.3.1 Validação do Compilador

4.3.1.1 Programas Válidos

O primeiro conjunto é formado por 14 programas válidos, numerados de 1 a 14. As Tabelas 20 e 21 resumem as construções da linguagem que cada programa exercita. Os códigos-fonte e os respectivos TACs gerados pelo compilador estão nos Códigos 3 a 30.

Tabela 20 – Programas de teste 1 a 7 e construções da linguagem exercitadas

N.º	**Programa**	**Construções exercitadas**
1	Soma de dois números	Declaração sem valor inicial, `RECEBE`, `MOSTRA` com texto e identificador, atribuição, operador `ADD`.
2	Par ou ímpar	Operador `RESTO`, operador relacional `IGUAL`, `SE` com `SENAO`.
3	Maior de dois números	Operador relacional `MAIOR`, `SE` com `SENAO`, `MOSTRA` com dois itens.
4	Número dentro de um intervalo	Operadores relacionais `MAG` e `MEG`, operador lógico `$$`, `SE` com `SENAO`.
5	Fatorial	Declaração com valor inicial, `ENQUANTO`, operadores `MEG` e `MULT`.
6	Fibonacci até N termos	`ENQUANTO` com várias variáveis, atribuições encadeadas, `MOSTRA` dentro do laço.
7	Contagem regressiva	`ENQUANTO` com operadores `MAIOR` e `SUB`, `MOSTRA` depois do laço.

Tabela 21 – Programas de teste 8 a 14 e construções da linguagem exercitadas

N.º	**Programa**	**Construções exercitadas**
8	Número primo	Tipo `LOGICO` com `VERDADEIRO` e `FALSO`, comando `PARE`, `SE` dentro de `ENQUANTO`, `SE` sem `SENAO`, condição formada por uma variável lógica, operadores `RESTO` e `MENOR`.
9	Soma dos dígitos	Operadores `DIV` e `RESTO`, `ENQUANTO`.
10	Tabuada	`PARA` com `PASSO`, operador `MULT`.
11	Tipos primitivos	Declarações dos quatro tipos (`TEXTO`, `DECIMAL`, `INTEIRO` e `LOGICO`), atribuição de uma comparação a uma variável lógica, `SE` cuja condição é essa variável.
12	Inteiros e decimais	Expressão que mistura `INTEIRO` e `DECIMAL`, com promoção do inteiro para decimal, e divisão inteira com `DIV`.
13	Operadores lógicos	Operadores lógicos `??` (ou) e `XX` (negação), `SE` com `SENAO`.
14	`PARA` com passo 2	Comentário de linha (`@@`) e laço `PARA` com passo diferente de 1.

As Tabelas 20 e 21 mostram que o conjunto cobre as três estruturas de controle da linguagem (condicional, `ENQUANTO` e `PARA`), o comando `PARE`, os comandos de entrada e saída, os quatro tipos primitivos, os cinco operadores aritméticos, seis operadores relacionais, os três operadores lógicos (`$$`, `??` e `XX`) e o comentário de linha. A seguir, cada programa é apresentado seguido do TAC que o compilador gerou para ele.

Código 3 – Programa de teste 1: soma de dois números

```
a[INTEIRO]#
b[INTEIRO]#
soma[INTEIRO]#
MOSTRA["Digite o primeiro numero: "]#
RECEBE[a]#
MOSTRA["Digite o segundo numero: "]#
RECEBE[b]#
soma VALE a ADD b#
MOSTRA["Soma: ", soma]#
```

Código 4 – TAC gerado para o programa de teste 1

```
print "Digite o primeiro numero: "
read a
print "Digite o segundo numero: "
read b
t1 = a + b
soma = t1
print "Soma: "
print soma
```

Código 5 – Programa de teste 2: par ou ímpar

```
n[INTEIRO]#
resto[INTEIRO]#
MOSTRA["Digite um numero: "]#
RECEBE[n]#
resto VALE n RESTO 2#
SE(resto IGUAL 0) {
    MOSTRA["Par"]#
}
SENAO {
    MOSTRA["Impar"]#
}
```

Código 6 – TAC gerado para o programa de teste 2

```
print "Digite um numero: "
read n
t1 = n % 2
resto = t1
if resto == 0 goto L1
t2 = 0
goto L2
L1:
t2 = 1
L2:
if t2 == 1 goto L3
goto L4
L3:
print "Par"
goto L5
L4:
print "Impar"
L5:
```

Código 7 – Programa de teste 3: maior de dois números

```
a[INTEIRO]#
b[INTEIRO]#
MOSTRA["Digite o primeiro numero: "]#
RECEBE[a]#
MOSTRA["Digite o segundo numero: "]#
RECEBE[b]#
SE(a MAIOR b) {
    MOSTRA["Maior: ", a]#
}
SENAO {
    MOSTRA["Maior: ", b]#
}
```

Código 8 – TAC gerado para o programa de teste 3

```
print "Digite o primeiro numero: "
read a
print "Digite o segundo numero: "
read b
if a > b goto L1
t1 = 0
goto L2
L1:
t1 = 1
L2:
if t1 == 1 goto L3
goto L4
L3:
print "Maior: "
print a
goto L5
L4:
print "Maior: "
print b
L5:
```

Código 9 – Programa de teste 4: número dentro de um intervalo

```
x[INTEIRO]#
a[INTEIRO]#
b[INTEIRO]#
MOSTRA["Digite x: "]#
RECEBE[x]#
MOSTRA["Digite o limite inferior: "]#
RECEBE[a]#
MOSTRA["Digite o limite superior: "]#
RECEBE[b]#
SE(x MAG a $$ x MEG b) {
    MOSTRA["Dentro do intervalo"]#
}
SENAO {
    MOSTRA["Fora do intervalo"]#
}
```

Código 10 – TAC gerado para o programa de teste 4

```
print "Digite x: "
read x
print "Digite o limite inferior: "
read a
print "Digite o limite superior: "
read b
if x >= a goto L1
t1 = 0
goto L2
L1:
t1 = 1
L2:
if x <= b goto L3
t2 = 0
goto L4
L3:
t2 = 1
L4:
t3 = t1 and t2
if t3 == 1 goto L5
goto L6
L5:
print "Dentro do intervalo"
goto L7
L6:
print "Fora do intervalo"
L7:
```

Código 11 – Programa de teste 5: fatorial

```
n[INTEIRO]#
i[INTEIRO]#
fatorial[INTEIRO] VALE 1#
MOSTRA["Digite um numero: "]#
RECEBE[n]#
i VALE 1#
ENQUANTO(i MEG n) {
    fatorial VALE fatorial MULT i#
    i VALE i ADD 1#
}
MOSTRA["Fatorial: ", fatorial]#
```

Código 12 – TAC gerado para o programa de teste 5

```
fatorial = 1
print "Digite um numero: "
read n
i = 1
L1:
if i <= n goto L4
t1 = 0
goto L5
L4:
t1 = 1
L5:
if t1 == 1 goto L2
goto L3
L2:
t2 = fatorial * i
fatorial = t2
t3 = i + 1
i = t3
goto L1
L3:
print "Fatorial: "
print fatorial
```

Código 13 – Programa de teste 6: Fibonacci até N termos

```
n[INTEIRO]#
i[INTEIRO]#
a[INTEIRO] VALE 0#
b[INTEIRO] VALE 1#
prox[INTEIRO]#
MOSTRA["Quantos termos: "]#
RECEBE[n]#
i VALE 1#
ENQUANTO(i MEG n) {
    MOSTRA[a]#
    prox VALE a ADD b#
    a VALE b#
    b VALE prox#
    i VALE i ADD 1#
}
```

Código 14 – TAC gerado para o programa de teste 6

```
a = 0
b = 1
print "Quantos termos: "
read n
i = 1
L1:
if i <= n goto L4
t1 = 0
goto L5
L4:
t1 = 1
L5:
if t1 == 1 goto L2
goto L3
L2:
print a
t2 = a + b
prox = t2
a = b
b = prox
t3 = i + 1
i = t3
goto L1
L3:
```

Código 15 – Programa de teste 7: contagem regressiva

```
n[INTEIRO]#
MOSTRA["Digite um numero: "]#
RECEBE[n]#
ENQUANTO(n MAIOR 0) {
    MOSTRA[n]#
    n VALE n SUB 1#
}
MOSTRA["Fim!"]#
```

Código 16 – TAC gerado para o programa de teste 7

```
print "Digite um numero: "
read n
L1:
if n > 0 goto L4
t1 = 0
goto L5
L4:
t1 = 1
L5:
if t1 == 1 goto L2
goto L3
L2:
print n
t2 = n - 1
n = t2
goto L1
L3:
print "Fim!"
```

Código 17 – Programa de teste 8: número primo

```
n[INTEIRO]#
i[INTEIRO]#
resto[INTEIRO]#
primo[LOGICO] VALE VERDADEIRO#
MOSTRA["Digite um numero: "]#
RECEBE[n]#
i VALE 2#
ENQUANTO(i MENOR n) {
    resto VALE n RESTO i#
    SE(resto IGUAL 0) {
        primo VALE FALSO#
        PARE#
    }
    i VALE i ADD 1#
}
SE(n MENOR 2) {
    primo VALE FALSO#
}
SE(primo) {
    MOSTRA["Primo"]#
}
SENAO {
    MOSTRA["Nao primo"]#
}
```

Código 18 – TAC gerado para o programa de teste 8

```
1  primo = VERDADEIRO
2  print "Digite um numero: "
3  read n
4  i = 2
5  L1:
6  if i < n goto L4
7  t1 = 0
8  goto L5
9  L4:
10 t1 = 1
11 L5:
12 if t1 == 1 goto L2
13 goto L3
14 L2:
15 t2 = n % i
16 resto = t2
17 if resto == 0 goto L6
18 t3 = 0
19 goto L7
20 L6:
21 t3 = 1
22 L7:
23 if t3 == 1 goto L8
24 goto L9
25 L8:
26 primo = FALSO
27 goto L3
28 goto L10
29 L9:
30 L10:
31 t4 = i + 1
32 i = t4
33 goto L1
34 L3:
35 if n < 2 goto L11
36 t5 = 0
37 goto L12
38 L11:
39 t5 = 1
40 L12:
41 if t5 == 1 goto L13
42 goto L14
43 L13:
44 primo = FALSO
45 goto L15
46 L14:
47 L15:
48 if primo == 1 goto L16
49 goto L17
50 L16:
51 print "Primo"
52 goto L18
53 L17:
54 print "Nao primo"
55 L18:
```

Código 19 – Programa de teste 9: soma dos dígitos

```
n[INTEIRO]#
digito[INTEIRO]#
soma[INTEIRO] VALE 0#
MOSTRA["Digite um numero: "]#
RECEBE[n]#
ENQUANTO(n MAIOR 0) {
    digito VALE n RESTO 10#
    soma VALE soma ADD digito#
    n VALE n DIV 10#
}
MOSTRA["Soma dos digitos: ", soma]#
```

Código 20 – TAC gerado para o programa de teste 9

```
soma = 0
print "Digite um numero: "
read n
L1:
if n > 0 goto L4
t1 = 0
goto L5
L4:
t1 = 1
L5:
if t1 == 1 goto L2
goto L3
L2:
t2 = n % 10
digito = t2
t3 = soma + digito
soma = t3
t4 = n / 10
n = t4
goto L1
L3:
print "Soma dos digitos: "
print soma
```

Código 21 – Programa de teste 10: tabuada

```
n[INTEIRO]#
i[INTEIRO]#
resultado[INTEIRO]#
MOSTRA["Digite um numero: "]#
RECEBE[n]#
PARA i DE 1 ATE 10 PASSO 1 {
    resultado VALE n MULT i#
    MOSTRA[resultado]#
}
```

Código 22 – TAC gerado para o programa de teste 10

```
print "Digite um numero: "
read n
i = 1
L1:
t1 = i <= 10
if t1 == 1 goto L2
goto L3
L2:
t2 = n * i
resultado = t2
print resultado
t3 = i + 1
i = t3
goto L1
L3:
```

Código 23 – Programa de teste 11: tipos primitivos

```
nome[TEXTO] VALE "Ana"#
altura[DECIMAL] VALE 1.75#
idade[INTEIRO] VALE 20#
maior[LOGICO] VALE FALSO#
maior VALE idade MAG 18#
MOSTRA["Nome: ", nome]#
MOSTRA["Altura: ", altura]#
MOSTRA["Idade: ", idade]#
SE(maior) {
    MOSTRA["Maior de idade"]#
}
SENAO {
    MOSTRA["Menor de idade"]#
}
```

Código 24 – TAC gerado para o programa de teste 11

```
nome = "Ana"
altura = 1.75
idade = 20
maior = FALSO
if idade >= 18 goto L1
t1 = 0
goto L2
L1:
t1 = 1
L2:
maior = t1
print "Nome: "
print nome
print "Altura: "
print altura
print "Idade: "
print idade
if maior == 1 goto L3
goto L4
L3:
print "Maior de idade"
goto L5
L4:
print "Menor de idade"
L5:
```

Código 25 – Programa de teste 12: expressões com inteiros e decimais

```
a[INTEIRO] VALE 7#
b[DECIMAL] VALE 2.5#
r[DECIMAL]#
c[INTEIRO]#
r VALE a ADD b#
c VALE a DIV 2#
MOSTRA["Soma decimal: ", r]#
MOSTRA["Divisao inteira: ", c]#
```

Código 26 – TAC gerado para o programa de teste 12

```
a = 7
b = 2.5
t1 = a + b
r = t1
t2 = a / 2
c = t2
print "Soma decimal: "
print r
print "Divisao inteira: "
print c
```

Código 27 – Programa de teste 13: operadores lógicos `??` e `XX`

```
x[INTEIRO] VALE 5#
ok[LOGICO]#
ok VALE x MAIOR 10 ?? x IGUAL 5#
SE(XX ok) {
    MOSTRA["ok falso"]#
}
SENAO {
    MOSTRA["ok verdadeiro"]#
}
```

Código 28 – TAC gerado para o programa de teste 13

```
x = 5
if x > 10 goto L1
t1 = 0
goto L2
L1:
t1 = 1
L2:
if x == 5 goto L3
t2 = 0
goto L4
L3:
t2 = 1
L4:
t3 = t1 or t2
ok = t3
t4 = not ok
if t4 == 1 goto L5
goto L6
L5:
print "ok falso"
goto L7
L6:
print "ok verdadeiro"
L7:
```

Código 29 – Programa de teste 14: comentário e `PARA` com passo 2

```
i[INTEIRO]#
s[INTEIRO] VALE 0#
@@ soma dos pares de 2 ate 10
PARA i DE 2 ATE 10 PASSO 2 {
    s VALE s ADD i#
}
MOSTRA["Soma: ", s]#
```

Código 30 – TAC gerado para o programa de teste 14

```
s = 0
i = 2
L1:
t1 = i <= 10
if t1 == 1 goto L2
goto L3
L2:
t2 = s + i
s = t2
t3 = i + 2
i = t3
goto L1
L3:
print "Soma: "
print s
```

Os TACs mostram o padrão de tradução descrito na Seção 3.6. Em todos eles, cada comparação usada como condição é traduzida pelas seis instruções que gravam 0 ou 1 em um temporário (programas 2 a 9, 11 e 13), exceto no laço `PARA`, em que a condição é calculada diretamente em uma instrução (programas 10 e 14). Cada `MOSTRA` com vários itens gera um `print` por item (programas 1, 3, 5, 9, 11, 12 e 14). No TAC do programa 8, a instrução `goto L10`, que vem logo depois de `goto L3`, nunca é executada: o primeiro desvio corresponde ao `PARE`, e o segundo é o desvio normal de fim do bloco `SE`. Esse código inalcançável não afeta a execução.

Os 14 programas foram aceitos em todas as fases: a análise léxica não reportou nenhum erro, os analisadores SLR e LL(1) aceitaram todos os programas e a análise semântica não reportou nenhum erro.

4.3.1.2 Programas com Erros Intencionais

O segundo conjunto é formado por 11 programas com um erro intencional cada, numerados de E1 a E11, que verificam os mecanismos de recuperação de erros em todas as fases. Os erros cobrem os três tipos: léxicos (E1 e E2), sintáticos (E3, E4 e E5) e semânticos (E6 a E11). Os códigos-fonte estão nos Códigos 31 e 32, e cada caso foi compilado separadamente.

Código 31 – Programas com erros léxicos e sintáticos (E1 a E5)

```
E1 (caractere invalido)
a[INTEIRO] VALE 3 & 2#
MOSTRA[a]#

E2 (texto sem fechar aspas)
MOSTRA["Ola]#

E3 (falta o delimitador #)
a[INTEIRO] VALE 3
MOSTRA[a]#

E4 (falta fechar o bloco com })
a[INTEIRO] VALE 1#
SE(a MAIOR 0) {
    MOSTRA[a]#

E5 (falta o parentese de abertura)
a[INTEIRO] VALE 1#
ENQUANTO a MAIOR 0) {
    a VALE a SUB 1#
}
```

Código 32 – Programas com erros semânticos (E6 a E11)

```
E6 (variavel nao declarada)
MOSTRA[x]#

E7 (tipo incompativel)
a[INTEIRO] VALE "texto"#

E8 (PARE fora de laco)
a[INTEIRO] VALE 1#
PARE#

E9 (redeclaracao)
a[INTEIRO]#
a[INTEIRO]#

E10 (condicao do SE nao logica)
n[INTEIRO] VALE 1#
SE(n ADD 1) {
    MOSTRA[n]#
}

E11 (decimal atribuido a inteiro)
a[INTEIRO]#
a VALE 2.5#
```

A Tabela 22 apresenta a quantidade de erros reportada por cada fase em modo de recuperação e a primeira mensagem de erro.

Tabela 22 – Resultado dos programas com erros intencionais

Caso	Léx.	SLR	LL(1)	Sem.	Primeira mensagem de erro
E1	1	10	1	0	Léxico: caractere inesperado `'&'`, linha 1, coluna 20
E2	1	1	2	0	Léxico: string não fechada
E3	0	1	5	0	Sintático: token inesperado `MOSTRA` na linha 2
E4	0	1	1	0	Sintático: token inesperado `$` (fim da entrada), esperado `}`
E5	0	10	10	0	Sintático: token inesperado `a` na linha 2, esperado `(`
E6	0	0	0	1	Linha 1: variável `'x'` não declarada em `MOSTRA`
E7	0	0	0	1	Linha 1: tipo incompatível, `'a'` é `INTEIRO`, expressão é `TEXTO`
E8	0	0	0	1	Linha 2: instrução `PARE` fora de estrutura de repetição
E9	0	0	0	1	Linha 2: variável `'a'` já declarada na linha 1
E10	0	0	0	1	Linha 2: condição do `SE` deve ser `LOGICO`, encontrado `INTEIRO`
E11	0	0	0	1	Linha 2: tipo incompatível, `'a'` é `INTEIRO`, expressão é `DECIMAL`

A Tabela 22 mostra que cada erro foi detectado pela fase correspondente à sua natureza e que as fases posteriores prosseguiram graças à recuperação. Nos casos E1 e E2, a análise léxica reportou o erro, descartou o trecho inválido e entregou os tokens restantes ao analisador sintático. Em E1, o caractere `&` foi descartado e o analisador sintático encontrou dois números consecutivos (`3 2`); em E2, o texto sem aspas de fechamento consome o restante da linha e o analisador sintático reporta o fim inesperado da entrada. Nos casos E3, E4 e E5, os erros são sintáticos e a análise semântica não reportou erros adicionais. Nos casos E6 a E11, os programas são sintaticamente corretos, e por isso os dois analisadores sintáticos os aceitaram e somente a análise semântica reportou o erro, com a linha e a regra violada.

Os números de erros sintáticos diferem entre os métodos porque as estratégias de recuperação são diferentes. Em E1 e E5, o analisador SLR chegou ao limite de 10 erros reportados, e em E5 o LL(1) também; em E3, o LL(1) reportou 5 erros e o SLR um. Isso mostra que um único erro de escrita pode produzir erros em cascata, e que o limite de 10 erros evita que o analisador fique indefinidamente em recuperação. Em todos os casos, ambos os métodos terminaram a análise e reportaram o erro real como primeira mensagem.

4.3.2 Validação da Berti VM

4.3.2.1 Execução do TAC Gerado pelo Compilador

Para cada um dos programas de teste 1 a 10, o TAC gerado pelo compilador (Códigos 4 a 22) foi carregado na VM sem nenhuma alteração. Cada programa foi executado com vários valores de entrada, incluindo valores limite, como zero e números negativos, o que resultou em 41 execuções. A saída esperada de cada execução foi determinada antes da execução, a partir do enunciado do programa, e comparada com a saída produzida pela VM. Nas tabelas, os valores de uma mesma saída aparecem separados por espaço em uma só linha, mas a VM escreve cada item de um `MOSTRA` e cada valor de um laço em uma linha própria, e os pedidos de entrada (como `Digite um numero:`) foram omitidos. As Tabelas 23 a 26 apresentam as execuções.

Tabela 23 – Execução dos programas de teste 1 a 3 na Berti VM

Prog.	**Entradas**	**Saída**
1	3, 4	`Soma: 7`
1	−5, 5	`Soma: 0`
1	0, 0	`Soma: 0`
2	7	`Impar`
2	10	`Par`
2	0	`Par`
2	−3	`Impar`
3	10, 4	`Maior: 10`
3	4, 10	`Maior: 10`
3	5, 5	`Maior: 5`

A Tabela 23 reúne os programas sem laços. As saídas coincidem com as esperadas em todas as execuções, inclusive nos valores limite: a soma com entradas negativas e nulas, o resto da divisão de um número negativo por 2 (a entrada −3 resulta em `Impar`, pois o resto da divisão tem o sinal do dividendo e, portanto, é −1, diferente de zero) e a comparação com entradas iguais no programa 3.

Tabela 24 – Execução dos programas de teste 4 e 5 na Berti VM

Prog.	**Entradas**	**Saída**
4	5, 1, 10	`Dentro do intervalo`
4	0, 1, 10	`Fora do intervalo`
4	11, 1, 10	`Fora do intervalo`
4	1, 1, 10	`Dentro do intervalo`
4	10, 1, 10	`Dentro do intervalo`
5	5	`Fatorial: 120`
5	0	`Fatorial: 1`
5	1	`Fatorial: 1`
5	10	`Fatorial: 3628800`

A Tabela 24 inclui o programa 4, em que as entradas 1 e 10 verificam os dois extremos do intervalo (os operadores `MAG` e `MEG` incluem os limites) e as entradas 0 e 11 verificam os valores imediatamente fora dele. No programa 5, as entradas 0 e 1 verificam que o laço não é executado, ou é executado uma vez, e que o resultado continua 1, e a entrada 10 verifica o resultado de várias multiplicações sucessivas (3628800).

Tabela 25 – Execução dos programas de teste 6 e 7 na Berti VM

Prog.	**Entradas**	**Saída**
6	6	`0 1 1 2 3 5`
6	1	`0`
6	0	(nenhum valor exibido)
6	10	`0 1 1 2 3 5 8 13 21 34`
7	3	`3 2 1 Fim!`
7	0	`Fim!`
7	1	`1 Fim!`
7	−2	`Fim!`

A Tabela 25 mostra, em cada linha, os valores da sequência separados por espaço. As entradas 0 do programa 6 e 0 e −2 do programa 7 verificam o caso em que o laço não é executado nenhuma vez: o programa 6 não exibe nenhum valor, e o programa 7 exibe somente a mensagem final. As entradas 1 verificam o laço executado uma única vez.

Tabela 26 – Execução dos programas de teste 8 a 10 na Berti VM

Prog.	**Entradas**	**Saída**
8	7	`Primo`
8	8	`Nao primo`
8	1	`Nao primo`
8	2	`Primo`
8	13	`Primo`
8	0	`Nao primo`
8	15	`Nao primo`
9	238	`Soma dos digitos: 13`
9	0	`Soma dos digitos: 0`
9	5	`Soma dos digitos: 5`
9	9999	`Soma dos digitos: 36`
10	4	`4 8 12 16 20 24 28 32 36 40`
10	0	`0 0 0 0 0 0 0 0 0 0`
10	−3	`-3 -6 -9 -12 -15 -18 -21 -24 -27 -30`

A Tabela 26 contém os programas mais complexos. O programa 8 exercita o comando `PARE`: a entrada 8 interrompe o laço na primeira iteração, pois 8 é divisível por 2, enquanto a entrada 13, um número primo, percorre todos os divisores. As entradas 0, 1 e 2 verificam os valores extremos do teste de primalidade, e a entrada 15 verifica um número composto cujo

primeiro divisor aparece depois da primeira iteração. No programa 9, a entrada 9999 verifica um número com quatro dígitos iguais. O programa 10, que usa o laço `PARA` com limites fixos, produz as saídas corretas também para o valor nulo e para o valor negativo.

Em todas as 41 execuções, a saída da VM foi igual à saída esperada.

4.3.2.2 Execução de Instruções do Formato Genérico

O segundo conjunto é formado por 5 programas escritos diretamente em TAC, que exercitam os recursos que a linguagem Bertol não emite: vetores, ponteiros, chamadas de função e as duas pilhas. Os programas estão nos Códigos 33 a 37.

Código 33 – Programa TAC 1: vetor, preenchimento e soma

```
i = 0
soma = 0
L1:
if i >= 5 goto L2
t1 = i + 1
t2 = t1 * 2
vet[i] = t2
i = i + 1
goto L1
L2:
i = 0
L3:
if i >= 5 goto L4
x = vet[i]
soma = soma + x
i = i + 1
goto L3
L4:
print "Soma do vetor:"
print soma
```

Código 34 – Programa TAC 2: ponteiros, troca de dois valores

```
a = 10
b = 20
pa = &a
pb = &b
t = *pa
u = *pb
*pa = u
*pb = t
print "a depois da troca:"
print a
print "b depois da troca:"
print b
```

Código 35 – Programa TAC 3: função de soma

```
x = 4
y = 5
param x
param y
r = call soma, 2
print "Resultado:"
print r
goto Lfim
soma:
s = x + y
return s
Lfim:
```

Código 36 – Programa TAC 4: chamadas aninhadas, soma de quadrados

```
x = 3
y = 4
param x
param y
r = call somaquad, 2
print "3^2 + 4^2 ="
print r
goto Lfim
somaquad:
param x
q1 = call quad, 1
x = y
param x
q2 = call quad, 1
res = q1 + q2
return res
quad:
qr = x * x
return qr
Lfim:
```

Código 37 – Programa TAC 5: vetor e função, maior elemento

```
vet[0] = 7
vet[1] = 42
vet[2] = 13
vet[3] = 28
n = 4
param n
m = call maior, 1
print "Maior elemento:"
print m
goto Lfim
maior:
mx = vet[0]
i = 1
Lw:
if i >= n goto Lret
v = vet[i]
if v <= mx goto Lnext
mx = v
Lnext:
i = i + 1
goto Lw
Lret:
return mx
Lfim:
```

Para cada programa, a saída esperada foi calculada manualmente antes da execução. A Tabela 27 apresenta os resultados.

Tabela 27 – Resultado dos programas em TAC com memória indexada e funções

N.º	**Recursos exercitados**	**Saída esperada e obtida**	**Estado final observado**
1	Escrita e leitura de vetor com posição calculada por variável	`Soma do vetor:` `30`	Vetor em `@1000` a `@1004`, com 2, 4, 6, 8 e 10

2	Ponteiros: &, leitura e escrita por *	`a depois da troca:`
		`20`
		`b depois da troca:`
		`10`
			`a` em `@1000` (valor 20) e `b` em `@101000` (valor 10)

3	Chamada com dois parâmetros e valor de retorno	`Resultado:` `9`	`r` = 9
4	Chamadas aninhadas e em sequência	`3^2 + 4^2 =` `25`	`q1` = 9, `q2` = 16, `r` = 25
5	Vetor com posição constante e variável, dentro de função	`Maior elemento:` `42`	Vetor em `@1000` a `@1003`, com 7, 42, 13 e 28; `m` = 42

A Tabela 27 mostra que a saída obtida coincidiu com a saída esperada nos cinco programas. A última coluna descreve o que foi observado nos painéis da VM ao final da execução. No

programa 1, os cinco elementos do vetor ocupam endereços consecutivos a partir de `@1000`, o que confirma o cálculo do endereço pela soma do endereço-base à posição. No programa 2, as variáveis `a` e `b` foram movidas para a memória indexada quando seus endereços foram tomados (`@1000` e `@101000`, blocos de 100000 endereços) e os valores trocaram corretamente pela escrita via ponteiro. No programa 3, a função recebeu os valores 4 e 5 e devolveu 9, que foi gravado na variável `r`. No programa 4, a função `somaquad` obteve 9 e 16 como quadrados de 3 e 4 e devolveu a soma, 25. No programa 5, a função percorre o vetor, que foi preenchido pelo código principal, o que confirma que os vetores são compartilhados entre o código principal e as funções.

4.3.2.3 Comparação entre os Modos de Execução

As 46 execuções realizadas na VM (41 dos programas de teste 1 a 10 e 5 dos programas em TAC) foram repetidas nos três modos, *Step*, *Run* e Instantâneo, o que totalizou 138 execuções. No modo *Run*, o intervalo entre as instruções foi ajustado para o mínimo, 1 milissegundo. Em todas as 138 execuções, a VM terminou no estado *finalizada*, sem nenhuma mensagem de erro, e a saída foi idêntica nos três modos para cada programa e conjunto de entradas. Esse resultado é relevante porque os três modos usam caminhos de controle diferentes, e as pausas e retomadas nas instruções `read` foram exercitadas nos três.

4.3.3 Síntese dos Resultados

A Tabela 28 resume os testes realizados.

Tabela 28 – Resumo dos casos de teste e resultados

Conjunto de testes	**Quantidade**	**Resultado**
Programas válidos (1 a 14) nas quatro fases do compilador	14 programas	Aceitos em todas as fases, sem erros
Programas com erros intencionais (E1 a E11)	11 programas	Erro detectado na fase esperada em todos os casos
Programas 1 a 10 executados na VM	41 execuções	Saída conforme a esperada em todas
Programas em TAC com memória indexada e funções	5 programas	Saída conforme a esperada em todos
Execuções nos três modos da VM	138 execuções	Saída idêntica nos três modos, sem erros de execução

A Tabela 28 mostra que todos os testes definidos foram executados com sucesso. O resultado mais relevante para os objetivos do trabalho é a execução dos TACs gerados pelo compilador, sem alterações, na Berti VM, com as saídas esperadas, o que indica que o ciclo da gramática à execução preserva a semântica dos programas.

5
Conclusão

Este trabalho apresentou o projeto e a implementação de um compilador para a linguagem Bertol, uma linguagem de programação imperativa com sintaxe em português estruturado, desenvolvida com fins didáticos, e de uma máquina virtual, a Berti VM, que executa o código intermediário gerado por esse compilador. O problema que motivou este trabalho decorre de uma lacuna observada no ensino de compiladores: na disciplina, o ciclo prático se encerra na geração de código intermediário, e o estudante que deseja executar o Código de Três Endereços (TAC) depende de uma ferramenta externa.

Para esse problema, foram desenvolvidos três artefatos integrados: a linguagem Bertol, um compilador completo para essa linguagem e a Berti VM. As quatro fases do compilador, análise léxica, análise sintática, análise semântica e geração de TAC, foram implementadas, com a análise sintática disponível nos métodos SLR e LL(1). Cada fase foi desenvolvida de modo a expor sua trajetória interna de execução, permitindo acompanhar o processo de compilação passo a passo, com modos de operação com e sem recuperação de erros. A Berti VM interpreta o TAC no navegador, instrução por instrução, e exibe o contador de programa, as variáveis, a memória indexada, as pilhas de chamadas e de parâmetros e a saída do programa, com execução passo a passo, contínua ou instantânea. Além das instruções emitidas pelo compilador, a VM aceita o formato genérico de TAC adotado na disciplina, com vetores, ponteiros, funções e conversões de tipo, o que permite executar também o código intermediário de outras linguagens desenvolvidas em sala de aula.

A validação foi realizada com casos de teste cobrindo as construções suportadas pela linguagem. No compilador, 14 programas válidos foram aceitos em todas as fases, e 11 programas com erros intencionais tiveram o erro detectado na fase esperada, com a análise prosseguindo graças à recuperação de erros. Na Berti VM, os TACs gerados pelo compilador para 10 programas foram executados sem nenhuma alteração, em 41 execuções com diferentes entradas, e produziram as saídas esperadas. Outros 5 programas escritos diretamente em TAC verificaram a memória

indexada e as chamadas de função. Todas essas execuções foram repetidas nos modos *Step*, *Run* e Instantâneo, num total de 138 execuções, com a mesma saída nos três modos e sem nenhum erro de execução.

Esses resultados indicam que o ciclo proposto, da gramática à execução, está completo: um programa escrito em Bertol é analisado, traduzido para TAC e executado, e a semântica do programa original é preservada ao longo das etapas. A principal contribuição deste trabalho é didática. Ela está em mostrar de forma detalhada e visual todo o processo de compilação, e, principalmente, em oferecer uma máquina virtual que permite ao estudante executar o código que o seu próprio compilador produz, tornando a execução parte do ciclo prático da disciplina de Compiladores.

Referências

AHO, A. V. et al. *Compiladores: princípios, técnicas e ferramentas*. 2. ed. São Paulo: Pearson Addison-Wesley, 2008. Citado 7 vezes nas páginas 12, 14, 15, 16, 17, 18 e 19.

ALAKÜLÄ, A. R.; FORS, N.; SÖDERBERG, E. Study of the use of property probes in an educational setting. *The Art, Science, and Engineering of Programming*, v. 10, n. 1, 2025. Citado 2 vezes nas páginas 20 e 21.

BEAZLEY, D. *PLY (Python Lex-Yacc)*. 2025. <https://github.com/dabeaz/ply>. Projeto descontinuado em dezembro de 2025. Acesso em: 21 jun. 2026. Citado 2 vezes nas páginas 20 e 21.

BOLL, D. C. C. *Além do Horizonte: tradução e execução de código de três endereços em Assembly MIPS*. Monografia (Trabalho de Conclusão de Curso (Bacharelado em Ciência da Computação)) — Universidade Estadual do Oeste do Paraná, Cascavel, 2023. Citado 2 vezes nas páginas 20 e 21.

Compiler Explorer. *Compiler Explorer*. 2025. <https://godbolt.org>. Acesso em: 21 jun. 2026. Citado 2 vezes nas páginas 20 e 21.

COOPER, K. D.; TORCZON, L. *Engineering a compiler*. 2. ed. Burlington: Morgan Kaufmann, 2012. Citado 2 vezes nas páginas 12 e 24.

CYTRON, R. et al. Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, v. 13, n. 4, p. 451–490, 1991. Citado na página 17.

GALANTE, G. *1-Introdução aos Compiladores*. 2025. Slides de aula da disciplina de Compiladores. Ciência da Computação, UNIOESTE, Cascavel. Citado na página 15.

GALANTE, G. *Código de Três Endereços: instruções, conversões de tipo e exemplos*. 2026. Material de apoio da disciplina de Compiladores. Ciência da Computação, UNIOESTE, Cascavel. Citado 2 vezes nas páginas 24 e 60.

GOLDBERG, R. P. Architecture of virtual machines. In: *AFIPS National Computer Conference*. New York: AFIPS Press, 1973. p. 309–318. Citado na página 19.

KC-ML2. *LLVM-FLOW: an open-source interactive visualization tool for comparing IR CFGs*. 2023. <https://github.com/kc-ml2/llvm-flow>. Apresentado no EuroLLVM Developers' Meeting 2023. Acesso em: 21 jun. 2026. Citado na página 20.

MERNIK, M.; ZUMER, V. An educational tool for teaching compiler construction. *IEEE Transactions on Education*, v. 46, n. 1, p. 61–68, 2003. Citado na página 12.

MOGENSEN, T. *Basics of compiler design*. Copenhagen: Department of Computer Science, University of Copenhagen, 2010. Citado na página 16.

NISAN, N.; SCHOCKEN, S. *The elements of computing systems: building a modern computer from first principles*. Cambridge: MIT Press, 2005. Citado 2 vezes nas páginas 13 e 20.

POPEK, G. J.; GOLDBERG, R. P. Formal requirements for virtualizable third generation architectures. *Communications of the ACM*, v. 17, n. 7, p. 412–421, 1974. Citado na página 19.

SEBESTA, R. W. *Conceitos de linguagens de programação*. 9. ed. Porto Alegre: Bookman, 2011. Citado na página 13.

SINGH, R. *Introduction to Compiler*. [S.l.]: New Age International (P) Limited, 2009. ISBN 9788122423983. Citado 4 vezes nas páginas 16, 17, 18 e 20.

SMITH, J. E.; NAIR, R. *Virtual machines: versatile platforms for systems and processes*. San Francisco: Morgan Kaufmann, 2005. Citado 2 vezes nas páginas 17 e 19.

SUNITHA, K. V. N. *Compiler Construction*. [S.l.]: Pearson, 2013. ISBN 9789332500297. Citado 2 vezes nas páginas 16 e 17.

YEGULALP, S. *What is LLVM? The power behind Swift, Rust, Clang, and more*. 2023. <https://www.infoworld.com/article/3247799/what-is-llvm-the-power-behind-swift-rust-clang-and-more.html>. Acesso em: 21 jun. 2026. Citado 2 vezes nas páginas 20 e 21.